

TOP v1.0  
Audition Ready Version  
Fully Integrated Core + Economic Module + DAC  
**Bundle Locked Version**

## Contents

|          |                                           |           |
|----------|-------------------------------------------|-----------|
| <b>1</b> | <b>Normative Status</b>                   | <b>9</b>  |
| 1.1      | Document Authority . . . . .              | 9         |
| 1.2      | Non-Extension Constraint . . . . .        | 9         |
| <b>2</b> | <b>Consensus Boundary</b>                 | <b>9</b>  |
| 2.1      | Canonical Input . . . . .                 | 9         |
| 2.2      | Canonical Event Log Definition . . . . .  | 9         |
| 2.3      | Total Ordering . . . . .                  | 10        |
| 2.4      | Consensus Output . . . . .                | 10        |
| 2.5      | Replay Contract . . . . .                 | 10        |
| 2.6      | Deterministic Completeness . . . . .      | 10        |
| <b>3</b> | <b>Canonical Numeric Domains</b>          | <b>10</b> |
| 3.1      | Signed Integer Domain . . . . .           | 10        |
| 3.2      | Fixed-Point Domain . . . . .              | 11        |
| 3.3      | Canonical Arithmetic Primitives . . . . . | 11        |
| 3.4      | Rounding Rule . . . . .                   | 11        |
| <b>4</b> | <b>Domain Separation Axioms</b>           | <b>11</b> |
| 4.1      | Explicit Promotion Rule . . . . .         | 11        |
| 4.2      | No Implicit Conversion . . . . .          | 12        |
| <b>5</b> | <b>Canonical Numerical Stabilizer</b>     | <b>12</b> |
| 5.1      | Definition . . . . .                      | 12        |
| 5.2      | Usage Sites . . . . .                     | 12        |
| 5.3      | Forbidden Constructions . . . . .         | 12        |

|           |                                                      |           |
|-----------|------------------------------------------------------|-----------|
| 5.4       | Determinism Consequence . . . . .                    | 13        |
| <b>6</b>  | <b>Integer Multiplication Safety Rule</b>            | <b>13</b> |
| <b>7</b>  | <b>Genesis Parameter Set <math>\Theta</math></b>     | <b>13</b> |
| 7.1       | Definition . . . . .                                 | 13        |
| 7.2       | Genesis Commitment to $\Theta$ (Normative) . . . . . | 13        |
| <b>8</b>  | <b>Universe Definitions (Genesis-Fixed)</b>          | <b>13</b> |
| 8.1       | SKU Universe . . . . .                               | 13        |
| 8.2       | Planet Universe . . . . .                            | 13        |
| <b>9</b>  | <b>Structural Parameters</b>                         | <b>14</b> |
| 9.1       | Horizon . . . . .                                    | 14        |
| 9.2       | Parent and Child Limits . . . . .                    | 14        |
| 9.3       | Temporal Decay . . . . .                             | 14        |
| 9.4       | Ripple Damping . . . . .                             | 14        |
| <b>10</b> | <b>Monetary Parameters (Fixed-Point)</b>             | <b>14</b> |
| 10.1      | Burn Weight . . . . .                                | 14        |
| 10.2      | Individual InvestMint Cap . . . . .                  | 14        |
| 10.3      | System InvestMint Cap . . . . .                      | 14        |
| 10.4      | Elasticity Bounds . . . . .                          | 14        |
| <b>11</b> | <b>Regulator Parameters</b>                          | <b>15</b> |
| 11.1      | Regulator Gain . . . . .                             | 15        |
| 11.2      | Smoothing Coefficient . . . . .                      | 15        |
| 11.3      | Target Branching . . . . .                           | 15        |
| <b>12</b> | <b>Stasis Parameters</b>                             | <b>15</b> |
| 12.1      | Accumulation Rate . . . . .                          | 15        |
| 12.2      | Base Realization Factor . . . . .                    | 15        |
| 12.3      | Target Mint . . . . .                                | 15        |
| <b>13</b> | <b>Economic Module Parameters</b>                    | <b>15</b> |
| 13.1      | Backlog Decay . . . . .                              | 15        |
| 13.2      | Supply Smoothing . . . . .                           | 16        |
| 13.3      | Utilization Weight . . . . .                         | 16        |
| 13.4      | Planetary Smoothing . . . . .                        | 16        |

|                                                              |           |
|--------------------------------------------------------------|-----------|
| 13.5 Planetary Stress Gain . . . . .                         | 16        |
| 13.6 Price Adjustment Gain . . . . .                         | 16        |
| 13.7 Log-Price Bounds . . . . .                              | 16        |
| 13.8 Workforce Adaptation . . . . .                          | 16        |
| 13.9 Scarcity Sensitivity . . . . .                          | 16        |
| 13.10 Planetary Multiplier Cap . . . . .                     | 16        |
| <b>14 Productivity and Impact Parameters (Genesis-Fixed)</b> | <b>17</b> |
| 14.1 Productivity Coefficients . . . . .                     | 17        |
| 14.2 Impact Coefficient Matrix . . . . .                     | 17        |
| <b>15 DAC Parameters</b>                                     | <b>17</b> |
| 15.1 Fixed-Point Precision . . . . .                         | 17        |
| 15.2 Logarithm Domain . . . . .                              | 17        |
| 15.3 Exponential Domain . . . . .                            | 18        |
| 15.4 Square Root Iterations . . . . .                        | 18        |
| 15.5 Lookup Tables . . . . .                                 | 18        |
| <b>16 -Lint Bound Parameters</b>                             | <b>18</b> |
| 16.1 Maximum Events per Epoch . . . . .                      | 18        |
| 16.2 Maximum Signers per Event . . . . .                     | 18        |
| 16.3 Maximum Burn . . . . .                                  | 18        |
| 16.4 Maximum Liquid Balance . . . . .                        | 18        |
| 16.5 Maximum Event Potential . . . . .                       | 19        |
| <b>17 Completeness Condition</b>                             | <b>19</b> |
| <b>18 Deterministic Arithmetic Contract (DAC)</b>            | <b>19</b> |
| 18.1 Purpose . . . . .                                       | 19        |
| 18.2 Logarithm Function . . . . .                            | 19        |
| 18.3 Exponential Function . . . . .                          | 20        |
| 18.4 Square Root Function (Optional) . . . . .               | 22        |
| 18.5 Reference Domain Recommendations . . . . .              | 22        |
| 18.6 DAC Determinism Theorem . . . . .                       | 23        |
| 18.7 Usage Sites in Protocol . . . . .                       | 23        |
| <b>19 Canonical Event Schema</b>                             | <b>23</b> |
| 19.1 Purpose . . . . .                                       | 23        |

|                                                       |           |
|-------------------------------------------------------|-----------|
| 19.2 Universal Event Fields . . . . .                 | 24        |
| 19.3 Type-Specific Fields . . . . .                   | 24        |
| 19.4 Event ID Computation . . . . .                   | 26        |
| 19.5 Event Admission Rules . . . . .                  | 26        |
| 19.6 Event Ordering in $\mathcal{L}$ . . . . .        | 27        |
| <b>20 Event Types (Consensus-Critical)</b>            | <b>27</b> |
| <b>21 Agent Universe Definition</b>                   | <b>27</b> |
| 21.1 Dynamic Agent Set . . . . .                      | 27        |
| 21.2 New Agent Initialization . . . . .               | 28        |
| <b>22 Epoch Candidate Set</b>                         | <b>28</b> |
| <b>23 Canonical Validation Procedure</b>              | <b>28</b> |
| 23.1 Burn Reservation Map . . . . .                   | 28        |
| 23.2 Validation Algorithm . . . . .                   | 28        |
| 23.3 Determinism Property . . . . .                   | 30        |
| <b>24 Apply Burn (H1)</b>                             | <b>30</b> |
| <b>25 Horizon Construction</b>                        | <b>30</b> |
| 25.1 Definition . . . . .                             | 30        |
| 25.2 Canonical Horizon ID List . . . . .              | 31        |
| <b>26 Canonical DAG Traversal</b>                     | <b>31</b> |
| 26.1 Acyclicity Precondition . . . . .                | 31        |
| 26.2 In-Degree Computation . . . . .                  | 31        |
| 26.3 Canonical Kahn Algorithm . . . . .               | 31        |
| 26.4 Ripple Order . . . . .                           | 32        |
| <b>27 Temporal Decay (Unified Definition)</b>         | <b>32</b> |
| <b>28 Fixed-Point Core Dynamics</b>                   | <b>33</b> |
| 28.1 Burn Promotion . . . . .                         | 33        |
| 28.2 Ripple Computation . . . . .                     | 33        |
| 28.3 Structural Intensity . . . . .                   | 33        |
| 28.4 Distributed Contribution Density (DCD) . . . . . | 34        |
| 28.5 Attribution Update . . . . .                     | 34        |

|                                                               |           |
|---------------------------------------------------------------|-----------|
| 28.6 State Storage Convention for Attribution . . . . .       | 34        |
| 28.7 Structural Determinism Theorem . . . . .                 | 34        |
| <b>29 Epoch Burn and Mint Budget</b>                          | <b>35</b> |
| 29.1 Epoch Burn . . . . .                                     | 35        |
| 29.2 Mint Budget . . . . .                                    | 35        |
| <b>30 InvestMint Processing</b>                               | <b>35</b> |
| 30.1 System Cap . . . . .                                     | 35        |
| 30.2 Individual Cap and Source Check . . . . .                | 36        |
| 30.3 Application . . . . .                                    | 36        |
| <b>31 Collateral Deposit Processing</b>                       | <b>37</b> |
| <b>32 Liquid Supply Update</b>                                | <b>37</b> |
| <b>33 Income Allocation</b>                                   | <b>38</b> |
| 33.1 Event Potential . . . . .                                | 38        |
| 33.2 Total Potential and Denominator . . . . .                | 38        |
| 33.3 Canonical Income Formula (Single Allowed Form) . . . . . | 38        |
| 33.4 H8 Income Split . . . . .                                | 39        |
| 33.5 Mint Conservation Identity . . . . .                     | 39        |
| 33.6 Monetary Determinism Theorem . . . . .                   | 40        |
| <b>34 Stasis Mechanism</b>                                    | <b>40</b> |
| 34.1 Rest Counter Update . . . . .                            | 40        |
| 34.2 Native Stasis Accumulation . . . . .                     | 40        |
| 34.3 Dynamic Stasis Realization Factor . . . . .              | 41        |
| 34.4 Stasis Realization . . . . .                             | 41        |
| 34.5 Auto-Liquidation on Production Events . . . . .          | 42        |
| <b>35 Regulator Layer</b>                                     | <b>43</b> |
| 35.1 Branching Degree . . . . .                               | 43        |
| 35.2 Smoothed Branching . . . . .                             | 43        |
| 35.3 Elasticity Update . . . . .                              | 44        |
| 35.4 Regulator Determinism Theorem . . . . .                  | 44        |
| <b>36 Economic Module</b>                                     | <b>45</b> |
| 36.1 Module Enablement . . . . .                              | 45        |

|                                                     |           |
|-----------------------------------------------------|-----------|
| 36.2 Lag Convention (Frozen) . . . . .              | 45        |
| 36.3 Backlog and Supply Update . . . . .            | 45        |
| 36.4 Scarcity . . . . .                             | 46        |
| 36.5 Capacity and Utilization . . . . .             | 46        |
| 36.6 Pressure . . . . .                             | 46        |
| 36.7 Planetary Stress . . . . .                     | 47        |
| 36.8 Planetary Multiplier . . . . .                 | 47        |
| 36.9 Log-Price Update . . . . .                     | 47        |
| 36.10 Workforce Allocation . . . . .                | 48        |
| 36.11 Economic Determinism Theorem . . . . .        | 49        |
| <b>37 Canonical Serialization and Hash Contract</b> | <b>49</b> |
| 37.1 Hash Function . . . . .                        | 49        |
| 37.2 Canonical SerializeState Definition . . . . .  | 49        |
| <b>38 Primitive Encodings</b>                       | <b>50</b> |
| 38.1 i128 Encoding . . . . .                        | 50        |
| 38.2 u32 Encoding . . . . .                         | 50        |
| 38.3 u8 Encoding . . . . .                          | 51        |
| 38.4 String Encoding . . . . .                      | 51        |
| 38.5 Bytes Encoding . . . . .                       | 51        |
| <b>39 Dense Map Encoding Rule</b>                   | <b>51</b> |
| 39.1 Canonical Dense Map Encoding . . . . .         | 52        |
| <b>40 Theta Binding</b>                             | <b>52</b> |
| <b>41 SerializeTheta</b>                            | <b>52</b> |
| 41.1 Structure . . . . .                            | 52        |
| 41.2 Parameter Encoding . . . . .                   | 52        |
| 41.3 Type Tags . . . . .                            | 53        |
| 41.4 Type-Specific Value Encodings . . . . .        | 53        |
| 41.5 Forbidden Formats . . . . .                    | 53        |
| <b>42 SerializeState Body</b>                       | <b>54</b> |
| 42.1 Section 1: Global Scalars . . . . .            | 54        |
| 42.2 Section 2: Agent Table . . . . .               | 54        |
| 42.3 Section 3: Horizon ID List . . . . .           | 54        |

|           |                                                       |           |
|-----------|-------------------------------------------------------|-----------|
| 42.4      | Section 4: Horizon Event Table . . . . .              | 55        |
| 42.5      | Section 5: Economic Tables . . . . .                  | 55        |
| 42.6      | Economic Module Always Enabled . . . . .              | 55        |
| <b>43</b> | <b>Serialization Determinism Theorem</b>              | <b>56</b> |
| <b>44</b> | <b>-Lint: Overflow Prevention</b>                     | <b>56</b> |
| 44.1      | Purpose . . . . .                                     | 56        |
| 44.2      | Event Layer Bounds . . . . .                          | 56        |
| 44.3      | Mint Bound . . . . .                                  | 57        |
| 44.4      | Income Multiplication Bound . . . . .                 | 57        |
| 44.5      | Ripple Bound . . . . .                                | 57        |
| 44.6      | Economic Bound . . . . .                              | 58        |
| 44.7      | DAC Domain Bounds . . . . .                           | 58        |
| 44.8      | Lint Validity Rule . . . . .                          | 58        |
| 44.9      | Recommended Lint Checker Output . . . . .             | 58        |
| 44.10     | -Lint Theorem . . . . .                               | 59        |
| <b>45</b> | <b>Canonical Epoch Execution Pipeline</b>             | <b>59</b> |
| 45.1      | Purpose . . . . .                                     | 59        |
| 45.2      | Pipeline Overview . . . . .                           | 59        |
| 45.3      | Step-by-Step Execution . . . . .                      | 60        |
| 45.4      | Pipeline Determinism Theorem . . . . .                | 66        |
| 45.5      | No Alternative Execution Orders . . . . .             | 67        |
| <b>46</b> | <b>Theorems and Bundle Lock</b>                       | <b>67</b> |
| 46.1      | Protocol Determinism Theorem . . . . .                | 67        |
| 46.2      | Replay Invariance Theorem . . . . .                   | 68        |
| 46.3      | No Hidden Degrees of Freedom . . . . .                | 68        |
| 46.4      | Bundle Lock Declaration . . . . .                     | 69        |
| 46.5      | Version Tag . . . . .                                 | 70        |
| 46.6      | Final Declaration . . . . .                           | 70        |
| <b>47</b> | <b>Audition Ready Master Context (Handover-Ready)</b> | <b>71</b> |
| 47.1      | Purpose of This Section . . . . .                     | 71        |
| 47.2      | Supersession Clause . . . . .                         | 71        |
| 47.3      | Consensus Boundary vs Tooling Boundary . . . . .      | 71        |

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| 47.4 Canonical Deployment Commitments . . . . .                        | 72        |
| 47.5 Remaining Work (Non-Negotiable for Audit-Grade Closure) . . . . . | 72        |
| 47.6 Work Agenda for the Next Session . . . . .                        | 73        |
| 47.7 Handover Package . . . . .                                        | 73        |
| <b>48 Final Declaration and Status</b>                                 | <b>74</b> |
| 48.1 Specification Status . . . . .                                    | 74        |
| 48.2 Closure Assessment . . . . .                                      | 74        |
| 48.3 Acknowledgment of Philosophical Foundation . . . . .              | 74        |



# 1 Normative Status

## 1.1 Document Authority

This document defines the complete, exclusive, and consensus-critical specification of TOP v1.0.

It supersedes all prior drafts, patches, engineering notes, and intermediate specifications.

Any divergence from this document constitutes a protocol fork.

## 1.2 Non-Extension Constraint

No section of this document may be modified without:

1. Explicit version increment (e.g., v3.2),
2. A new canonical document,
3. A new  $\Theta$  commitment hash,
4. Updated test vectors,
5. Updated hash commitments.

Interpretive freedom is prohibited.

# 2 Consensus Boundary

## 2.1 Canonical Input

**Definition 1** (Consensus Input). *The canonical input to TOP v1.0 is the ordered pair:*

$$(\Theta, \mathcal{L})$$

*where:*

- $\Theta$  is the immutable genesis parameter set,
- $\mathcal{L}$  is the canonical accepted event log.

## 2.2 Canonical Event Log Definition

**Definition 2** (Canonical Event Log).  $\mathcal{L}$  contains exactly those events that have been accepted by the protocol validation rules at their respective epochs.

*Rejected or invalid events are never part of  $\mathcal{L}$ .*

There exists no alternative interpretation of  $\mathcal{L}$ .

## 2.3 Total Ordering

The event log  $\mathcal{L}$  MUST satisfy total ordering:

$$e_i < e_j \iff \left( epoch(e_i) < epoch(e_j) \right) \vee \left( epoch(e_i) = epoch(e_j) \wedge id(e_i) < id(e_j) \right)$$

All implementations MUST process events in this order.

## 2.4 Consensus Output

For each epoch  $T$ , the protocol produces:

- Post-state  $\mathcal{S}_T$ ,
- Canonical serialization  $\text{SerializeState}(\mathcal{S}_T)$ ,
- Canonical state hash  $H_T$ .

## 2.5 Replay Contract

**Theorem 1** (Replay Contract). *Given identical  $(\Theta, \mathcal{L})$ , all conformant implementations MUST produce identical:*

$$\mathcal{S}_T \text{ and } H_T \quad \forall T.$$

## 2.6 Deterministic Completeness

TOP v1.0 is deterministically complete if and only if:

- All arithmetic is explicitly defined,
- All iteration orders are explicit,
- All map universes are explicit,
- All truncations are explicit,
- All overflow behavior is explicit,
- All rounding rules are explicit.

This document guarantees deterministic completeness.

# 3 Canonical Numeric Domains

## 3.1 Signed Integer Domain

**Definition 3** (i128 Domain). *All consensus-critical integers are represented as signed 128-bit two's-complement integers:*

$$x \in [-2^{127}, 2^{127} - 1].$$

Overflow MUST cause deterministic abort. Wrapping and saturation are strictly forbidden.

### 3.2 Fixed-Point Domain

**Definition 4** (Fixed-Point Representation). *Let  $k \in \Theta$  be the fixed-point precision exponent. Define:*

$$\text{FPONE} := 10^k.$$

*A fixed-point value  $x_{fp} \in \mathbb{Z}$  represents:*

$$x = \frac{x_{fp}}{\text{FPONE}}.$$

All non-trivial arithmetic in TOP v1.0 operates in fixed-point integer form.

Floating-point hardware or libraries are prohibited.

### 3.3 Canonical Arithmetic Primitives

**Definition 5** (Fixed-Point Multiplication).

$$\text{mul\_fp}(a_{fp}, b_{fp}) := \left\lfloor \frac{a_{fp} \cdot b_{fp}}{\text{FPONE}} \right\rfloor$$

**Definition 6** (Fixed-Point Division).

$$\text{div\_fp}(a_{fp}, b_{fp}) := \left\lfloor \frac{a_{fp} \cdot \text{FPONE}}{b_{fp}} \right\rfloor, \quad b_{fp} > 0.$$

All intermediate multiplications MUST be i128-checked. Division by zero or negative denominator MUST abort.

### 3.4 Rounding Rule

All divisions in consensus-critical arithmetic use floor rounding:

$$\left\lfloor \frac{a}{b} \right\rfloor.$$

No alternative rounding mode is permitted.

## 4 Domain Separation Axioms

### 4.1 Explicit Promotion Rule

Promotion from base integer  $x \in \mathbb{Z}$  to fixed-point is:

$$x_{fp} := x \cdot \text{FPONE}.$$

This is the only permitted base-to-fixed conversion.

## 4.2 No Implicit Conversion

Strictly forbidden:

- Adding base-integer values to fixed-point values.
- Multiplying base-integer values by fixed-point parameters without explicit promotion.
- Implicit division by FPONE.
- Mixing units without explicit scaling.

All unit transitions MUST appear explicitly in formulas.

## 5 Canonical Numerical Stabilizer

### 5.1 Definition

**Definition 7** (Numerical Stabilizer). *The protocol defines a single stabilizer parameter:*

$$\varepsilon_{\text{num},fp} \in \Theta, \quad \varepsilon_{\text{num},fp} > 0.$$

*It is a fixed-point integer.*

There exists no base-integer version of  $\varepsilon_{\text{num}}$ .

### 5.2 Usage Sites

The stabilizer appears exclusively in:

1. Attribution denominator:  $Z_T^{fp} + \varepsilon_{\text{num},fp}$ ,
2. Income denominator:  $\sum P_T^{fp}(e) + \varepsilon_{\text{num},fp}$ ,
3. Softmax denominator:  $\sum \exp_{fp}(\cdot) + \varepsilon_{\text{num},fp}$ ,
4. Scarcity denominator:  $\bar{S}_T(g) + \varepsilon_{\text{num},fp}$ ,
5. Utilization denominator:  $O_T^{fp}(g) + \varepsilon_{\text{num},fp}$ .

No other usage is permitted.

### 5.3 Forbidden Constructions

Strictly prohibited:

- Multiplying  $\varepsilon_{\text{num},fp}$  by FPONE.
- Dividing  $\varepsilon_{\text{num},fp}$  by FPONE.
- Introducing any base-integer stabilizer.
- Conditional denominator replacement logic.

## 5.4 Determinism Consequence

Because  $\varepsilon_{\text{num},fp} > 0$ , all denominators in which it appears are strictly positive.

Division-by-zero is impossible.

No runtime branching on zero denominators is permitted.

## 6 Integer Multiplication Safety Rule

All integer multiplications in consensus-critical code MUST satisfy:

$$|a \cdot b| < 2^{127}.$$

If overflow occurs, execution MUST abort.

Widened intermediate precision (e.g., 256-bit) is not part of the protocol definition.

Overflow safety is enforced through  $\Theta$ -Lint bounds (Section 44).

## 7 Genesis Parameter Set $\Theta$

### 7.1 Definition

**Definition 8** (Genesis Parameter Set).  *$\Theta$  is the immutable, consensus-critical parameter set that fully determines all deterministic computations not contained in the event log  $\mathcal{L}$ .*

$\Theta$  MUST be serialized and hash-bound to the protocol state (see Serialization section).

### 7.2 Genesis Commitment to $\Theta$ (Normative)

A TOP v1.0 deployment MUST publish the genesis commitment hash  $H_\Theta$  as part of the protocol instance definition.

Nodes MUST refuse execution unless the locally loaded  $\Theta$  yields exactly this published  $H_\Theta$ .

## 8 Universe Definitions (Genesis-Fixed)

### 8.1 SKU Universe

$$\mathcal{G} := [g_1, \dots, g_{N_G}]$$

Finite lexicographically sorted list of SKU identifiers, stored in  $\Theta$ .

### 8.2 Planet Universe

$$\mathcal{P} := [p_1, \dots, p_{N_P}]$$

Finite lexicographically sorted list of planetary keys, stored in  $\Theta$ .

These universes are immutable once defined in  $\Theta$ .

## 9 Structural Parameters

### 9.1 Horizon

$$H \in \mathbb{N}, \quad H \geq 1$$

Number of epochs included in rolling horizon.

### 9.2 Parent and Child Limits

$$\begin{aligned} K_{parents} &\in \mathbb{N}, & K_{parents} &\geq 1 \\ K_{children} &\in \mathbb{N}, & K_{children} &\geq 1 \end{aligned}$$

### 9.3 Temporal Decay

$$\tau \in \mathbb{N}, \quad \tau \geq 1$$

Decay time constant (in epochs).

### 9.4 Ripple Damping

$$\gamma_{fp} \in \mathbb{Z}, \quad 0 \leq \gamma_{fp} \leq \text{FPONE}$$

Ripple propagation damping factor (fixed-point).

## 10 Monetary Parameters (Fixed-Point)

### 10.1 Burn Weight

$$\lambda_{b,fp} \in \mathbb{Z}, \quad \lambda_{b,fp} \geq 0$$

Weight given to direct burn in event potential.

### 10.2 Individual InvestMint Cap

$$\phi_{0,fp} \in \mathbb{Z}, \quad \phi_{0,fp} \geq 0$$

Individual InvestMint cap as fraction of liquid supply.

### 10.3 System InvestMint Cap

$$\theta_{0,fp} \in \mathbb{Z}, \quad \theta_{0,fp} \geq 0$$

System-wide InvestMint cap as fraction of mint budget.

### 10.4 Elasticity Bounds

$$\epsilon_{\min,fp}, \epsilon_{\max,fp} \in \mathbb{Z}, \quad \epsilon_{\min,fp} \leq \epsilon_{\max,fp}$$

Bounds on regulator elasticity parameter.

## 11 Regulator Parameters

### 11.1 Regulator Gain

$$\kappa_{P,fp} \in \mathbb{Z}, \quad \kappa_{P,fp} \geq 0$$

Proportional gain for elasticity adjustment.

### 11.2 Smoothing Coefficient

$$\beta_{fp} \in \mathbb{Z}, \quad 0 \leq \beta_{fp} \leq \text{FPONE}$$

EMA coefficient for smoothed branching.

### 11.3 Target Branching

$$\sigma_{target,fp} \in \mathbb{Z}$$

Target average branching degree (fixed-point).

## 12 Stasis Parameters

### 12.1 Accumulation Rate

$$\rho_{fp} \in \mathbb{Z}, \quad \rho_{fp} \geq 0$$

Native stasis accumulation rate.

### 12.2 Base Realization Factor

$$\eta_{S,0,fp} \in \mathbb{Z}, \quad \eta_{S,0,fp} \geq 0$$

Base realization factor (before dynamic scaling).

### 12.3 Target Mint

$$M_{target} \in \mathbb{Z}, \quad M_{target} > 0$$

Reference mint budget for stasis realization scaling.

## 13 Economic Module Parameters

### 13.1 Backlog Decay

$$\delta_{B,fp} \in \mathbb{Z}, \quad 0 \leq \delta_{B,fp} \leq \text{FPONE}$$

Backlog decay rate per epoch.

### 13.2 Supply Smoothing

$$\xi_{fp} \in \mathbb{Z}, \quad 0 \leq \xi_{fp} \leq \text{FPONE}$$

EMA coefficient for smoothed supply.

### 13.3 Utilization Weight

$$\lambda_{u,fp} \in \mathbb{Z}, \quad \lambda_{u,fp} \geq 0$$

Weight of underutilization in pressure computation.

### 13.4 Planetary Smoothing

$$\zeta_{fp} \in \mathbb{Z}, \quad 0 \leq \zeta_{fp} \leq \text{FPONE}$$

EMA coefficient for planetary stress.

### 13.5 Planetary Stress Gain

$$\kappa_{X,fp} \in \mathbb{Z}, \quad \kappa_{X,fp} \geq 0$$

Gain mapping planetary stress to price multiplier.

### 13.6 Price Adjustment Gain

$$\kappa_{p,fp} \in \mathbb{Z}$$

Gain for log-price adjustment (may be negative).

### 13.7 Log-Price Bounds

$$\ell_{\min,fp}, \ell_{\max,fp} \in \mathbb{Z}, \quad \ell_{\min,fp} \leq \ell_{\max,fp}$$

Bounds on log-price to prevent extreme values.

### 13.8 Workforce Adaptation

$$\nu_{fp} \in \mathbb{Z}, \quad 0 \leq \nu_{fp} \leq \text{FPONE}$$

Workforce migration rate per epoch.

### 13.9 Scarcity Sensitivity

$$\alpha_{\chi,fp} \in \mathbb{Z}$$

Weight of scarcity in workforce utility (may be negative).

### 13.10 Planetary Multiplier Cap

$$\mu_{\max,fp} \in \mathbb{Z}, \quad \mu_{\max,fp} \geq \text{FPONE}$$

Upper bound on planetary multiplier.



## 14 Productivity and Impact Parameters (Genesis-Fixed)

### 14.1 Productivity Coefficients

$$A_{fp} : \mathcal{G} \rightarrow \mathbb{Z}, \quad A_{fp}(g) \geq 0 \quad \forall g \in \mathcal{G}$$

Productivity coefficient determines capacity per unit workforce allocation.

Encoded in  $\Theta$  as dense map over  $\mathcal{G}$ , lexicographically ordered.

Type tag in `SerializeTheta`: 0x05 (`map(str→i128)`).

This parameter is genesis-fixed and does not update during protocol execution.

### 14.2 Impact Coefficient Matrix

$$imp_{fp} : \mathcal{G} \times \mathcal{P} \rightarrow \mathbb{Z}$$

The impact coefficient matrix defines the planetary impact per unit of confirmed supply for each SKU-planet pair.

All entries are fixed-point integers.

**Encoding in  $\Theta$ .** Encoded as nested map structure:

- Outer map:  $\mathcal{G} \rightarrow \text{Map}(\mathcal{P} \rightarrow \mathbb{Z})$
- Keys lexicographically sorted at both levels
- Dense over  $\mathcal{G} \times \mathcal{P}$

Type tag in `SerializeTheta`: 0x05 (`map`).

**Access notation.**  $imp_{fp}(g, p)$  denotes the impact of SKU  $g$  on planet key  $p$ .

## 15 DAC Parameters

### 15.1 Fixed-Point Precision

$$k \in \mathbb{N}$$

Exponent determining  $\text{FPONE} = 10^k$ .

### 15.2 Logarithm Domain

$$x_{\min, fp} > 0, \quad x_{\max, fp} > x_{\min, fp}$$

Domain bounds for logarithm function.

$$N_{\log} \in \mathbb{N}, \quad N_{\log} \geq 2$$

Number of grid points in log table.

### 15.3 Exponential Domain

$$u_{\min,fp}, u_{\max,fp} \in \mathbb{Z}, \quad u_{\min,fp} < u_{\max,fp}$$

Domain bounds for exponential function.

$$N_{\text{exp}} \in \mathbb{N}, \quad N_{\text{exp}} \geq 2$$

Number of grid points in exp table.

### 15.4 Square Root Iterations

$$N_{\sqrt{\cdot} \in \mathbb{N}}, \quad N_{\sqrt{\cdot} \geq 1}$$

Fixed number of Newton-Raphson iterations for square root (if needed).

### 15.5 Lookup Tables

$$\text{log\_table} : [0, N_{\text{log}} - 1] \rightarrow \mathbb{Z}$$

$$\text{exp\_table} : [0, N_{\text{exp}} - 1] \rightarrow \mathbb{Z}$$

Precomputed tables of exact i128 values.

The exact integer table values are part of  $\Theta$  and consensus-critical.

## 16 -Lint Bound Parameters

### 16.1 Maximum Events per Epoch

$$N_{E,\text{max}} \in \mathbb{N}$$

### 16.2 Maximum Signers per Event

$$N_{\text{sig},\text{max}} \in \mathbb{N}$$

### 16.3 Maximum Burn

$$b_{\text{max}} \in \mathbb{N}$$

### 16.4 Maximum Liquid Balance

$$E_{\text{max}} \in \mathbb{N}$$

## 16.5 Maximum Event Potential

$$P_{\max}^{fp} \in \mathbb{Z}$$

Explicit bound on event potential magnitude for overflow prevention.

All -Lint bounds MUST satisfy constraints defined in the -Lint section (Section 44).

## 17 Completeness Condition

**Proposition 1** (Completeness).  *$\Theta$  is complete if and only if:*

$$(\Theta, \mathcal{L}) \Rightarrow \text{unique } S_T \quad \forall T.$$

## 18 Deterministic Arithmetic Contract (DAC)

### 18.1 Purpose

The DAC provides consensus-critical fixed-point implementations of:

- $\log_{fp}(x_{fp})$  — natural logarithm
- $\exp_{fp}(u_{fp})$  — exponential function
- $\sqrt{_{fp}(x_{fp})}$  — square root (optional)

All implementations MUST produce identical results given identical inputs and  $\Theta$ .

### 18.2 Logarithm Function

#### Domain Definition

The logarithm is defined over the domain:

$$x_{\min,fp} \leq x_{fp} \leq x_{\max,fp}$$

where  $x_{\min,fp}, x_{\max,fp} \in \Theta$  are fixed-point parameters.

Both bounds MUST satisfy  $x_{\min,fp} > 0$ .

#### Grid Parameters

$\Theta$  contains:

- $N_{\log} \in \mathbb{N}$  — number of grid points ( $N_{\log} \geq 2$ )
- Grid spacing:

$$\Delta_{\log,fp} = \left\lfloor \frac{x_{\max,fp} - x_{\min,fp}}{N_{\log} - 1} \right\rfloor$$

where  $\Delta_{\log,fp} > 0$  MUST be verified during  $\Theta$ -Lint.

## Lookup Table

$\Theta$  contains the precomputed table:

$$\text{log\_table} : [0, N_{\log} - 1] \rightarrow \mathbb{Z}$$

The exact integer table values are part of  $\Theta$  and consensus-critical.

Tables **MUST** be generated offline using high-precision arithmetic (e.g., Python `mpmath` with  $\geq 128$  bits precision) and rounded to nearest i128.

## Canonical Algorithm

Given input  $x_{fp}$ :

### 1. Clamp input:

$$x'_{fp} = \begin{cases} x_{\min,fp} & \text{if } x_{fp} < x_{\min,fp} \\ x_{\max,fp} & \text{if } x_{fp} > x_{\max,fp} \\ x_{fp} & \text{otherwise} \end{cases}$$

### 2. Compute grid index (base integer):

$$i = \left\lfloor \frac{x'_{fp} - x_{\min,fp}}{\Delta_{\log,fp}} \right\rfloor$$

### 3. Clamp index:

$$i = \begin{cases} 0 & \text{if } i < 0 \\ N_{\log} - 2 & \text{if } i \geq N_{\log} - 1 \\ i & \text{otherwise} \end{cases}$$

### 4. Compute base grid point:

$$x_i = x_{\min,fp} + i \cdot \Delta_{\log,fp}$$

### 5. Compute fractional offset (fixed-point):

$$f_{fp} = \text{div\_fp}(x'_{fp} - x_i, \Delta_{\log,fp})$$

where  $0 \leq f_{fp} \leq \text{FPONE}$ .

### 6. Linear interpolation:

$$\log_{fp}(x_{fp}) = \text{log\_table}[i] + \text{mul\_fp}(f_{fp}, \text{log\_table}[i+1] - \text{log\_table}[i])$$

All arithmetic **MUST** be i128-checked.

No alternative algorithm is permitted.

## 18.3 Exponential Function

### Domain Definition

The exponential is defined over the domain:

$$u_{\min,fp} \leq u_{fp} \leq u_{\max,fp}$$

where  $u_{\min,fp}, u_{\max,fp} \in \Theta$  are fixed-point parameters.

## Grid Parameters

$\Theta$  contains:

- $N_{\text{exp}} \in \mathbb{N}$  — number of grid points ( $N_{\text{exp}} \geq 2$ )
- Grid spacing:

$$\Delta_{\text{exp},fp} = \left\lfloor \frac{u_{\text{max},fp} - u_{\text{min},fp}}{N_{\text{exp}} - 1} \right\rfloor$$

where  $\Delta_{\text{exp},fp} > 0$  MUST be verified during  $\Theta$ -Lint.

## Lookup Table

$\Theta$  contains:

$$\text{exp\_table} : [0, N_{\text{exp}} - 1] \rightarrow \mathbb{Z}$$

The exact integer table values are part of  $\Theta$  and consensus-critical.

## Canonical Algorithm

Given input  $u_{fp}$ :

### 1. Clamp input:

$$u'_{fp} = \begin{cases} u_{\text{min},fp} & \text{if } u_{fp} < u_{\text{min},fp} \\ u_{\text{max},fp} & \text{if } u_{fp} > u_{\text{max},fp} \\ u_{fp} & \text{otherwise} \end{cases}$$

### 2. Compute grid index:

$$j = \left\lfloor \frac{u'_{fp} - u_{\text{min},fp}}{\Delta_{\text{exp},fp}} \right\rfloor$$

### 3. Clamp index:

$$j = \begin{cases} 0 & \text{if } j < 0 \\ N_{\text{exp}} - 2 & \text{if } j \geq N_{\text{exp}} - 1 \\ j & \text{otherwise} \end{cases}$$

### 4. Compute base grid point:

$$u_j = u_{\text{min},fp} + j \cdot \Delta_{\text{exp},fp}$$

### 5. Compute fractional offset:

$$f_{fp} = \text{div\_fp}(u'_{fp} - u_j, \Delta_{\text{exp},fp})$$

### 6. Linear interpolation:

$$\text{exp}_{fp}(u_{fp}) = \text{exp\_table}[j] + \text{mul\_fp}(f_{fp}, \text{exp\_table}[j + 1] - \text{exp\_table}[j])$$

All arithmetic MUST be i128-checked.

## 18.4 Square Root Function (Optional)

If square root is required, use Newton-Raphson iteration:

### Canonical Algorithm

Given  $x_{fp} \geq 0$ :

1. **Initial guess:**

$$y_0 = x_{fp}$$

2. **Iterate  $N_{\sqrt{\cdot}}$  times** (where  $N_{\sqrt{\cdot} \in \Theta}$ ):

$$y_{k+1} = \left\lfloor \frac{y_k + \text{div\_fp}(x_{fp}, y_k)}{2} \right\rfloor$$

3. **Return:**

$$\sqrt{_{fp}(x_{fp})} = y_{N_{\sqrt{\cdot}}}$$

The number of iterations  $N_{\sqrt{\cdot}}$  is consensus-critical and MUST be identical across all implementations.

Recommended:  $N_{\sqrt{\cdot}=10}$  (provides 30 bits of precision).

## 18.5 Reference Domain Recommendations

For typical TOP deployments:

### Logarithm Domain.

- $x_{\min,fp} = 0.001 \cdot \text{FPONE}$  (log of very small values)
- $x_{\max,fp} = 1000 \cdot \text{FPONE}$  (log of large values)
- $N_{\log} = 1000$  (grid density 0.1% of range)

This covers  $\log(0.001) \approx -6.9$  to  $\log(1000) \approx 6.9$ .

### Exponential Domain.

- $u_{\min,fp} = -10 \cdot \text{FPONE}$  (exp of negative values for decay)
- $u_{\max,fp} = 10 \cdot \text{FPONE}$  (exp of positive values for growth)
- $N_{\exp} = 1000$

This covers  $\exp(-10) \approx 0.0000454$  to  $\exp(10) \approx 22026$ .

These are recommendations only; actual values are deployment-specific and MUST be declared in  $\Theta$ .

## 18.6 DAC Determinism Theorem

**Theorem 2** (DAC Determinism). *Given identical:*

- $\Theta$  (including *log\_table*, *exp\_table*),
- input value  $x_{fp}$  or  $u_{fp}$ ,

*all conformant implementations produce identical:*

$$\log_{fp}(x_{fp}), \quad \exp_{fp}(u_{fp}), \quad \sqrt{_{fp}(x_{fp})}$$

*Proof.* All algorithms are:

- deterministic integer arithmetic,
- fixed iteration order,
- fixed rounding (floor division),
- table-driven with no platform-dependent computation,
- bounded by explicit clipping.

No floating-point arithmetic. No branching on tolerances. Therefore outputs are unique.

□

## 18.7 Usage Sites in Protocol

DAC functions appear in:

- **Temporal Decay:**  $\lambda_{T,fp}(e, y) = \exp_{fp}(u_{fp})$  where  $u_{fp} = -\lfloor \Delta \cdot \text{FPONE} / \tau \rfloor$
- **Structural Intensity:**  $\sigma_T^{fp}(e) = \log_{fp}((1 + |\text{Children}(e)|) \cdot \text{FPONE})$
- **Stasis Accumulation:**  $\log_{fp}((1 + \text{rest}_T(a)) \cdot \text{FPONE})$
- **Log-Price Update:**  $\ell_T^{fp}(g)$  includes  $\log_{fp}(\mu_{T-1}^{fp}(g))$
- **Price Field:**  $p_T^{fp}(g) = \exp_{fp}(\ell_T^{fp}(g))$
- **Softmax (Workforce):**  $\exp_{fp}(a_T^{fp}(g) - m_T)$

All usage MUST call the canonical DAC functions defined here.

No alternative implementations (e.g., Taylor series, CORDIC) are permitted.

## 19 Canonical Event Schema

### 19.1 Purpose

This section defines the consensus-critical structure of events that may enter the canonical log  $\mathcal{L}$ .

All event field extraction and validation MUST follow these rules.

## 19.2 Universal Event Fields

Every event  $e$  MUST contain:

**Event ID.**

$$id(e) \in \text{ASCII string}$$

Must be unique across all events. Lexicographic ordering is consensus-critical.

**Epoch.**

$$epoch(e) \in \mathbb{N}$$

The epoch in which the event is submitted.

**Event Type.**

$$type(e) \in \{\text{STANDARD, INVESTMINT, COLLATERAL\_DEPOSIT, DEMAND, CONFIRM}\}$$

**Signers.**

$$Signers(e) := [a_1, \dots, a_n]$$

Non-empty list of agent IDs (ASCII strings), lexicographically sorted.

Constraint:  $n \geq 1$  and  $n \leq N_{sig,max}$ .

**Parents.**

$$Parents(e) := [e_1, \dots, e_m]$$

List of parent event IDs (ASCII strings), lexicographically sorted.

Constraint:  $m \leq K_{parents}$ .

May be empty (orphan events).

**Burn Amount.**

$$b(e) \in \mathbb{Z}$$

Base-integer burn amount.

For INVESTMINT and COLLATERAL\_DEPOSIT:  $b(e) = 0$ .

For STANDARD, DEMAND, CONFIRM:  $b(e) > 0$ .

## 19.3 Type-Specific Fields

### STANDARD

No additional required fields.

May contain optional metadata (non-consensus).



## INVESTMINT

### Delta Stasis.

$$\Delta S(e) \in \mathbb{Z}, \quad \Delta S(e) > 0$$

Amount of stasis to convert to liquid.

### Single Signer Constraint.

$$|\text{Signers}(e)| = 1$$

## COLLATERAL\_DEPOSIT

### Deposit Amount.

$$\text{deposit}(e) \in \mathbb{Z}, \quad \text{deposit}(e) > 0$$

Declared fiat deposit amount.

### Single Signer Constraint.

$$|\text{Signers}(e)| = 1$$

## DEMAND

### SKU.

$$\text{sku}(e) \in \mathcal{G}$$

The requested SKU identifier.

### Quantity.

$$\text{quantity}(e) \in \mathbb{Z}, \quad \text{quantity}(e) > 0$$

Demanded quantity (base integer).

## CONFIRM

### SKU.

$$\text{sku}(e) \in \mathcal{G}$$

The supplied SKU identifier.

### Quantity.

$$\text{quantity}(e) \in \mathbb{Z}, \quad \text{quantity}(e) > 0$$

Confirmed supply quantity (base integer).

## 19.4 Event ID Computation

The event ID  $id(e)$  is computed deterministically as:

$$id(e) := \text{lowercase hex}(\text{SHA256}(\text{EventContent}(e)))$$

where  $\text{EventContent}(e)$  is the canonical serialization of all consensus-critical fields in fixed order:

1.  $\text{u32LE}(\text{epoch}(e))$
2.  $\text{u8}(\text{type\_tag})$  where type tags are:
  - $0x01 = \text{STANDARD}$
  - $0x02 = \text{INVESTMINT}$
  - $0x03 = \text{COLLATERAL\_DEPOSIT}$
  - $0x04 = \text{DEMAND}$
  - $0x05 = \text{CONFIRM}$
3.  $\text{u32LE}(|\text{Signers}(e)|)$  followed by  $\text{str}(a_1) \parallel \dots \parallel \text{str}(a_n)$
4.  $\text{u32LE}(|\text{Parents}(e)|)$  followed by  $\text{str}(e_1) \parallel \dots \parallel \text{str}(e_m)$
5.  $\text{i128LE}(b(e))$
6. Type-specific fields in fixed order:
  - INVESTMINT:  $\text{i128LE}(\Delta S(e))$
  - COLLATERAL\_DEPOSIT:  $\text{i128LE}(\text{deposit}(e))$
  - DEMAND/CONFIRM:  $\text{str}(sku(e)) \parallel \text{i128LE}(\text{quantity}(e))$
  - STANDARD: (no additional fields)

The resulting SHA-256 digest (32 bytes) is rendered as lowercase hexadecimal (64 characters).

This ID computation is consensus-critical.

**Collision Resistance.** SHA-256 provides  $2^{256}$  output space. Event ID collisions are cryptographically infeasible.

**Immutability.** Once an event enters  $\mathcal{L}$ , its ID is fixed. Event content cannot be modified post-acceptance.

## 19.5 Event Admission Rules

An event  $e$  may be submitted to epoch  $T$  if:

- $\text{epoch}(e) = T$
- All fields satisfy type constraints above
- All parent IDs reference events in  $\mathcal{L}$

- All signer IDs are valid ASCII strings
- SKU identifiers (if present) are in  $\mathcal{G}$

Submission does not guarantee acceptance.

Events are accepted into  $\mathcal{L}$  if and only if they pass validation (see Canonical Validation Procedure).

## 19.6 Event Ordering in $\mathcal{L}$

Events in  $\mathcal{L}$  are totally ordered by:

$$(epoch(e), id(e))$$

Primary key: increasing epoch.

Secondary key: lexicographic ordering of  $id(e)$  (as ASCII string).

This order is consensus-critical and immutable.

**Storage and Consensus.** The storage format of  $\mathcal{L}$  and the consensus mechanism by which events are admitted to  $\mathcal{L}$  are implementation-specific and outside the scope of this SSOT.

This specification defines only the state transition function given an accepted log  $\mathcal{L}$ .

## 20 Event Types (Consensus-Critical)

The protocol recognizes exactly five event types:

- STANDARD — general structural participation
- INVESTMINT — convert stasis to liquid (single signer only)
- COLLATERAL\_DEPOSIT — declare fiat deposit (single signer only)
- DEMAND — request SKU supply
- CONFIRM — confirm SKU supply

No additional event types are permitted.

Type-specific validation constraints are enforced during validation (Step V1).

## 21 Agent Universe Definition

### 21.1 Dynamic Agent Set

$$\mathcal{A}_T = \text{sorted unique set of all signers appearing in } \mathcal{L} \text{ up to epoch } T.$$

Agents are admitted dynamically as they sign events.

The Agent table is dense over  $\mathcal{A}_T$ .

All agents in  $\mathcal{A}_T$  MUST appear in serialization.

## 21.2 New Agent Initialization

When an agent  $a$  first appears as a signer in  $\mathcal{L}$ , and does not exist in the pre-state  $\mathcal{S}_{T-1}$ , the agent is implicitly initialized with:

$$E_{T-1}(a) := 0, \quad S_{T-1}^{native}(a) := 0, \quad S_{T-1}^{coll}(a) := 0, \quad rest_{T-1}(a) := 0$$

This initialization is consensus-critical and occurs during validation reads.

No genesis defaults from  $\Theta$  are used.

All agents start with zero balances and zero rest.

## 22 Epoch Candidate Set

For epoch  $T$ :

$$\mathcal{E}_T^{cand} = \{e \in \text{external input} \mid epoch(e) = T\}.$$

Before validation,  $\mathcal{E}_T^{cand}$  is sorted lexicographically by  $id(e)$ .

## 23 Canonical Validation Procedure

Validation reads exclusively from  $\mathcal{S}_{T-1}$ .

No state mutation occurs during validation.

### 23.1 Burn Reservation Map

Initialize:

$$R_{burn}(a) := 0 \quad \forall a \in \mathcal{A}_{T-1}.$$

If a signer  $a$  is not in  $\mathcal{A}_{T-1}$ , it is treated as new agent with implicit zero initialization.

### 23.2 Validation Algorithm

Process each  $e \in \mathcal{E}_T^{cand}$  in lexicographic order of  $id(e)$ .

#### Step V1 — Structural + Type Checks

Reject  $e$  if:

- $Signers(e) = \emptyset$
- $|Signers(e)| > N_{sig,max}$
- $|Parents(e)| > K_{parents}$
- Type-specific rules violated:
  - STANDARD: reject if  $b(e) \leq 0$
  - DEMAND: reject if  $b(e) \leq 0$  or  $sku(e) \notin \mathcal{G}$  or  $quantity(e) \leq 0$
  - CONFIRM: reject if  $b(e) \leq 0$  or  $sku(e) \notin \mathcal{G}$  or  $quantity(e) \leq 0$
  - INVESTMINT: reject if  $b(e) \neq 0$  or  $\Delta S(e) \leq 0$  or  $|Signers(e)| \neq 1$
  - COLLATERAL\_DEPOSIT: reject if  $b(e) \neq 0$  or  $deposit(e) \leq 0$  or  $|Signers(e)| \neq 1$

### Step V2 — Parent Validity

Reject  $e$  if any parent  $p \in Parents(e)$ :

- is not in  $\mathcal{L}$
- has  $epoch(p) \geq T$

Future parents and non-existent parents are forbidden.

### Step V3 — Canonical Burn Split (H7)

Let  $Signers(e) = [a_1, \dots, a_n]$  sorted lexicographically.

$$q = \left\lfloor \frac{b(e)}{n} \right\rfloor, \quad r = b(e) - nq.$$

$$b_{a_i}(e) = \begin{cases} q + r & i = 1 \\ q & i > 1 \end{cases}$$

This split is consensus-critical.

The lexicographically first signer receives the remainder.

### Step V4 — Pre-Check Reservation (No Rollback)

For each signer  $a$  require:

$$E_{T-1}(a) - R_{burn}(a) \geq b_a(e).$$

If violated for any signer: reject  $e$  permanently.

No rollback occurs.

## Step V5 — Reservation Update

If all checks pass:

$$R_{burn}(a) \leftarrow R_{burn}(a) + b_a(e) \quad \forall a \in Signers(e)$$

Append  $e$  to  $\mathcal{E}_T^{val}$ .

### 23.3 Determinism Property

Because:

- candidates are processed in lexicographic order,
- no rollback occurs,
- no backtracking occurs,
- reservation is cumulative,

the validated set  $\mathcal{E}_T^{val}$  is uniquely determined.

## 24 Apply Burn (H1)

After validation completes:

For each  $e \in \mathcal{E}_T^{val}$  in lexicographic order:

$$E_T(a) \leftarrow E_T(a) - b_a(e) \quad \forall a \in Signers(e)$$

Burn is debited exactly once.

No re-check is permitted.

No second subtraction is permitted.

All  $E_T(a)$  values MUST remain non-negative (guaranteed by validation).

## 25 Horizon Construction

### 25.1 Definition

$$T_{\min} = \max(0, T - H + 1).$$

$$\mathcal{H}_T = \{e \in \mathcal{L} \mid T_{\min} \leq epoch(e) \leq T\}.$$

Horizon is derived exclusively from the accepted canonical log  $\mathcal{L}$ .

There is no alternative source.

## 25.2 Canonical Horizon ID List

$HIds_T$  = lexicographically sorted list of  $id(e)$  for  $e \in \mathcal{H}_T$ .

This list is used for deterministic iteration over horizon events.

## 26 Canonical DAG Traversal

### 26.1 Acyclicity Precondition

The horizon graph is acyclic because:

$$epoch(parent) < epoch(child).$$

Edges within the same epoch are forbidden by validation (Step V2).

Therefore the directed graph over  $\mathcal{H}_T$  is a DAG.

### 26.2 In-Degree Computation

For each  $e \in \mathcal{H}_T$ :

$$\text{indegree}(e) = |\{u \in \mathcal{H}_T \mid e \in Parents(u)\}|.$$

All counts MUST fit in i128.

This is guaranteed by  $\Theta$ -Lint bounds on  $N_{E,\max}$  and  $K_{children}$ .

### 26.3 Canonical Kahn Algorithm

Define zero-indegree set:

$$Z = \{e \in \mathcal{H}_T \mid \text{indegree}(e) = 0\}$$

Sort  $Z$  lexicographically by  $id(e)$ .

Initialize empty list  $L$ .

While  $Z \neq \emptyset$ :

1. Remove lexicographically smallest  $e$  from  $Z$ .
2. Append  $e$  to  $L$ .
3. For each child  $y$  of  $e$  (in lexicographic order of  $id(y)$ ):
  - decrement  $\text{indegree}(y)$
  - if  $\text{indegree}(y) = 0$ , insert  $y$  into  $Z$  (maintaining sorted order)

After loop terminates:

$$|L| = |\mathcal{H}_T|$$

If not, abort with DAG violation error (cycle detected).

## 26.4 Ripple Order

$$\text{RippleOrder} = \text{reverse}(L).$$

All Ripple computations MUST follow this order.

No DFS traversal is permitted.

No container-dependent traversal is permitted.

No alternative topological sort is permitted.

This is the only canonical traversal order.

## 27 Temporal Decay (Unified Definition)

For parent-child pair  $(e, y)$  where  $e \in \text{Parents}(y)$ :

$$\Delta = \text{epoch}(y) - \text{epoch}(e), \quad \Delta \geq 1.$$

Define exponent argument (base integer):

$$u_{\text{raw}} = - \left\lfloor \frac{\Delta \cdot \text{FPONE}}{\tau} \right\rfloor.$$

Promote to fixed-point and clip:

$$u_{fp} = u_{\text{raw}} \cdot \text{FPONE}$$

$$u_{fp} = \text{clip}(u_{fp}, u_{\min, fp}, u_{\max, fp}).$$

Decay factor:

$$\lambda_{T, fp}(e, y) = \exp_{fp}(u_{fp}).$$

This is the only permitted decay definition.

All parent-child decay factors are computed before Ripple traversal.



## 28 Fixed-Point Core Dynamics

### 28.1 Burn Promotion

For each  $e \in \mathcal{H}_T$ :

$$b_{fp}(e) := b(e) \cdot \text{FPONE}.$$

Promotion from base integer to fixed-point.

### 28.2 Ripple Computation

For each  $e$  in RippleOrder (children before parents):

$$R_T^{fp}(e) = b_{fp}(e) + \sum_{y \in \text{Children}(e)} \text{mul\_fp}(\gamma_{fp}, C_{fp}(e, y))$$

where:

$$C_{fp}(e, y) = \text{mul\_fp}\left(\lambda_{T,fp}(e, y), \left\lfloor \frac{\text{mul\_fp}(R_T^{fp}(y), \alpha_T^{fp}(y))}{|\text{Parents}(y)|} \right\rfloor\right).$$

**Children Set.**

$$\text{Children}(e) = \{y \in \mathcal{H}_T \mid e \in \text{Parents}(y)\}$$

sorted lexicographically by  $id(y)$  for deterministic iteration.

**Attribution Read.** For  $y \in \mathcal{H}_T$ :

If  $y \in \mathcal{H}_{T-1}$ , read  $\alpha_T^{fp}(y) := \alpha_{T-1}^{stored}(y)$  from pre-state.

If  $y \in \mathcal{E}_T^{val}$  (new event this epoch), then  $\alpha_T^{fp}(y) := 0$ .

All arithmetic is fixed-point.

All intermediate results MUST be i128-checked.

### 28.3 Structural Intensity

For each  $e \in \mathcal{H}_T$ :

$$\sigma_T^{fp}(e) = \log_{fp}((1 + |\text{Children}(e)|) \cdot \text{FPONE}).$$

Uses DAC logarithm function.

## 28.4 Distributed Contribution Density (DCD)

For each  $e \in \mathcal{H}_T$ :

$$DCD_T^{fp}(e) = \text{mul\_fp}(\sigma_T^{fp}(e), R_T^{fp}(e)).$$

DCD combines structural position with causal mass.

## 28.5 Attribution Update

### Total Structural Mass

$$Z_T^{fp} = \sum_{e \in \mathcal{H}_T} DCD_T^{fp}(e).$$

All summations MUST be i128-checked.

### Canonical Attribution

For each  $e \in \mathcal{H}_T$ :

$$\alpha_{T+1}^{fp}(e) = \text{div\_fp}\left(DCD_T^{fp}(e), Z_T^{fp} + \varepsilon_{\text{num}, fp}\right).$$

No conditional branching permitted.

Denominator is strictly positive by construction.

## 28.6 State Storage Convention for Attribution

**Definition 9** (Attribution Storage Rule). *The state  $\mathcal{S}_T$  stores:*

$$\alpha_T^{\text{stored}}(e) = \alpha_{T+1}^{fp}(e).$$

Therefore:

Epoch  $T + 1$  reads  $\alpha_T^{\text{stored}}$  as its  $\alpha_T^{fp}$  input.

There is no dual naming.

This removes index ambiguity.

## 28.7 Structural Determinism Theorem

**Theorem 3.** *Given identical  $(\Theta, \mathcal{L})$ , the structural core produces:*

$$R_T^{fp}, \quad DCD_T^{fp}, \quad \alpha_T^{\text{stored}}$$

*uniquely for all  $T$ .*

*Proof.* All traversal ordering is canonical (Kahn algorithm with lexicographic tie-breaking).

All arithmetic is fixed-point with floor rounding.

No branching on data-dependent tolerance exists.

Attribution reads follow storage convention deterministically.

Therefore outputs are unique.

□

## 29 Epoch Burn and Mint Budget

### 29.1 Epoch Burn

$$B_T = \sum_{e \in \mathcal{E}_T^{val}} b(e)$$

Sum over all validated events (base integer).

All summations MUST be i128-checked.

### 29.2 Mint Budget

Let  $\epsilon_{T-1}^{fp}$  be the regulator elasticity stored in  $\mathcal{S}_{T-1}$ .

Define:

$$M_T = B_T + \left\lfloor \frac{\epsilon_{T-1}^{fp} \cdot B_T}{\text{FPONE}} \right\rfloor.$$

This is the only permitted mint formula.

No floating-point arithmetic is allowed.

$M_T$  is the total income budget available for distribution this epoch.

## 30 InvestMint Processing

### 30.1 System Cap

$$Cap_T^{sys} = \left\lfloor \frac{\theta_{0,fp} \cdot M_T}{\text{FPONE}} \right\rfloor.$$

System-wide cap on total InvestMint conversions this epoch.

InvestMint candidates are processed in lexicographic id order.

Initialize:

$$CapRem_T^{sys} := Cap_T^{sys}.$$

Filter InvestMint events:

$$\mathcal{M}_T = \{e \in \mathcal{E}_T^{val} \mid \text{type}(e) = \text{INVESTMINT}\}$$

sorted lexicographically by  $\text{id}(e)$ .

For each  $m \in \mathcal{M}_T$  in order:

- If  $\Delta S(m) \leq \text{CapRem}_T^{sys}$ :

$$\text{CapRem}_T^{sys} \leftarrow \text{CapRem}_T^{sys} - \Delta S(m)$$

Proceed to individual cap check.

- Else: Reject  $m$  permanently (no partial fill).

Let  $\mathcal{M}_T^{acc}$  be the set of system-cap-accepted InvestMint events.

### 30.2 Individual Cap and Source Check

For each accepted  $m \in \mathcal{M}_T^{acc}$  with single signer  $a$ :

Compute individual cap:

$$\text{Cap}_{T-1}^{ind} = \left\lfloor \frac{\phi_{0,fp} \cdot M_{T-1}^{liq}}{\text{FPONE}} \right\rfloor.$$

where  $M_{T-1}^{liq}$  is the liquid supply from pre-state.

Require:

$$\Delta S(m) \leq \text{Cap}_{T-1}^{ind}.$$

Require sufficient stasis source:

$$\Delta S(m) \leq S_{T-1}^{native}(a) + S_{T-1}^{coll}(a).$$

If any check fails, reject  $m$  (remove from  $\mathcal{M}_T^{acc}$ ).

### 30.3 Application

For each fully accepted  $m \in \mathcal{M}_T^{acc}$ :

Deduct native stasis first:

$$\Delta S_{nat} = \min(\Delta S(m), S_{T-1}^{native}(a)).$$

$$\Delta S_{coll} = \Delta S(m) - \Delta S_{nat}.$$

Update stasis:

$$S_T^{native}(a) = S_T^{native}(a) - \Delta S_{nat}.$$

$$S_T^{coll}(a) = S_T^{coll}(a) - \Delta S_{coll}.$$

Credit liquid:

$$E_T(a) = E_T(a) + \Delta S(m).$$

All updates are deterministic.

Native stasis is depleted before collateral stasis.

## 31 Collateral Deposit Processing

For each accepted `COLLATERAL_DEPOSIT` event  $e \in \mathcal{E}_T^{val}$  with single signer  $a$ :

$$S_T^{coll}(a) := S_T^{coll}(a) + deposit(e)$$

where  $deposit(e)$  is the declared fiat deposit amount.

**External Verification (Non-Consensus).** The protocol does not verify fiat deposits on-chain.

External bridge operators **MUST** verify fiat receipt before accepting deposit events into  $\mathcal{L}$ .

**Redemption Rule.** Collateral stasis can be used for InvestMint (auto or explicit).

InvestMint from collateral reduces redeemable fiat 1:1.

Maximum redeemable fiat at any time:

$$F_{redeem}(a) = S_T^{coll}(a)$$

Redemption mechanics are external to this protocol specification.

## 32 Liquid Supply Update

After all InvestMint processing and before income allocation:

$$M_T^{liq} = M_{T-1}^{liq} + \sum_{e \in \mathcal{E}_T^{val}} I_T(e) + \sum_{m \in \mathcal{M}_T^{acc}} \Delta S(m) - B_T$$

where:

- $M_{T-1}^{liq}$  is liquid supply from pre-state
- $M_T$  is epoch mint budget

- $\sum_{m \in \mathcal{M}_T^{acc}} \Delta S(m)$  is total explicit InvestMint liquidation
- $B_T$  is total epoch burn

Note: Auto-liquidation is added separately after auto-liquidation processing.

All arithmetic MUST be i128-checked.

This value MUST be serialized in Global Scalars.

## 33 Income Allocation

### 33.1 Event Potential

For each  $e \in \mathcal{E}_T^{val}$ :

$$P_T^{fp}(e) = DCD_T^{fp}(e) + \text{mul\_fp}(\lambda_{b,fp}, b(e) \cdot \text{FPONE}).$$

Event potential combines structural contribution (DCD) with direct burn.

For events not in  $\mathcal{H}_T$  (orphans or outside horizon):

$DCD_T^{fp}(e) := 0$  (not computed for non-horizon events).

### 33.2 Total Potential and Denominator

$$\Sigma_T^{fp} = \sum_{e \in \mathcal{E}_T^{val}} P_T^{fp}(e).$$

$$Den_T^{fp} = \Sigma_T^{fp} + \varepsilon_{\text{num},fp}.$$

Denominator is strictly positive.

No conditional replacement is permitted.

### 33.3 Canonical Income Formula (Single Allowed Form)

Income is defined mathematically as:

$$I_T(e) = \left\lfloor \frac{M_T \cdot P_T^{fp}(e)}{Den_T^{fp}} \right\rfloor.$$

### Binding Implementation Order

The only permitted evaluation sequence:

$$N := M_T \cdot P_T^{fp}(e)$$

$$I_T(e) := \left\lfloor \frac{N}{Den_T^{fp}} \right\rfloor.$$

Rules:

- Multiplication MUST be i128-checked.
- If overflow occurs, abort deterministically.
- No 256-bit widening is permitted.
- No reordering such as  $\text{div\_fp}(P, Den) \cdot M_T$  is permitted.

This resolves all floor-order ambiguity.

### 33.4 H8 Income Split

Let signers of  $e$  be:

$$\text{Signers}(e) = [a_1, \dots, a_n]$$

sorted lexicographically.

$$q = \left\lfloor \frac{I_T(e)}{n} \right\rfloor, \quad r = I_T(e) - nq.$$

$$I_{a_i}(e) = \begin{cases} q + r & i = 1 \\ q & i > 1 \end{cases}$$

The lexicographically first signer receives the remainder.

No alternative split is permitted.

Apply to balances:

$$E_T(a_i) := E_T(a_i) + I_{a_i}(e) \quad \forall i \in [1, n]$$

### 33.5 Mint Conservation Identity

$$\sum_{e \in \mathcal{E}_T^{val}} I_T(e) \leq M_T.$$

The total remainder caused by flooring is bounded by:

$$|\mathcal{E}_T^{val}|.$$

No redistribution of remainder is permitted.

Unallocated mint (if any) is permanently lost.

### 33.6 Monetary Determinism Theorem

**Theorem 4.** *Given identical  $(\Theta, \mathcal{L})$ , the monetary layer produces identical:*

$$M_T, \quad Cap_T^{sys}, \quad \mathcal{M}_T^{acc}, \quad I_T(e), \quad E_T(a).$$

*Proof.* All arithmetic is fixed-point integer arithmetic.

All evaluation order is canonical.

All ordering is lexicographic.

No floating arithmetic.

No branch-based denominator substitution.

InvestMint cap enforcement is deterministic with lexicographic ordering.

Income split uses deterministic floor division with fixed remainder assignment.

Therefore all outputs are unique.

□

## 34 Stasis Mechanism

### 34.1 Rest Counter Update

For each agent  $a \in \mathcal{A}_T$ :

Define production indicator:

$$\text{produced}_T(a) := \begin{cases} 1 & \text{if } \exists e \in \mathcal{E}_T^{val} : a \in \text{Signers}(e) \wedge \text{type}(e) \in \{\text{STANDARD}, \text{CONFIRM}\} \\ 0 & \text{otherwise} \end{cases}$$

Update rest counter:

$$\text{rest}_T(a) := \begin{cases} 0 & \text{if } \text{produced}_T(a) = 1 \\ \text{rest}_{T-1}(a) + 1 & \text{otherwise} \end{cases}$$

Rest counter measures epochs of non-production.

DEMAND, INVESTMINT, and COLLATERAL\_DEPOSIT events do NOT reset rest.

Only STANDARD and CONFIRM (productive events) reset rest to zero.

### 34.2 Native Stasis Accumulation

For each agent  $a \in \mathcal{A}_T$ :

$$S_T^{\text{native}}(a) = S_{T-1}^{\text{native}}(a) + \text{mul\_fp}(\rho_{fp}, \log_{fp}((1 + \text{rest}_T(a)) \cdot \text{FPONE}))$$

All arithmetic MUST be i128-checked.



Stasis accumulation is uncapped (no maximum).

Native stasis represents accumulated potential from rest periods.

### 34.3 Dynamic Stasis Realization Factor

Compute average planetary stress:

$$X_{avg,T}^{fp} = \left\lfloor \frac{\sum_{p \in \mathcal{P}} X_T^{fp}(p)}{|\mathcal{P}|} \right\rfloor$$

Note: This uses  $X_T^{fp}(p)$  which is computed in the Economic Module before this step (see Canonical Epoch Pipeline).

Compute activity factor:

$$activity_T^{fp} = \text{div\_fp}(M_T \cdot \text{FPONE}, M_{target})$$

Compute stress factor:

$$stress_T^{fp} = \text{FPONE} + X_{avg,T}^{fp}$$

Effective realization factor:

$$\eta_{S,fp,T}^{eff} = \text{mul\_fp}(\eta_{S,0,fp}, \text{div\_fp}(activity_T^{fp}, stress_T^{fp}))$$

where  $\eta_{S,0,fp} \in \Theta$  is the base realization parameter.

The effective factor scales with system activity and inversely with planetary stress.

### 34.4 Stasis Realization

Total structural mass for this epoch:

$$\Sigma_T^{DCD} = \sum_{e \in \mathcal{E}_T^{val}} DCD_T^{fp}(e)$$

Maximum realizable stasis per agent:

$$S_{max}^{real}(a, T) = \min \left( S_T^{native}(a), \text{mul\_fp}(\eta_{S,fp,T}^{eff}, \Sigma_T^{DCD}) \right)$$

Realization is capped by both:

- Available native stasis
- System-wide structural activity (via DCD sum)

Deduct realized portion from native stasis (regardless of liquidation outcome):

$$S_T^{native}(a) := S_T^{native}(a) - S_{max}^{real}(a, T)$$

This deduction occurs immediately after realization computation.

Realized stasis becomes available for auto-liquidation.

### 34.5 Auto-Liquidation on Production Events

Auto-liquidation shares the remaining system InvestMint cap  $CapRem_T^{sys}$  after explicit InvestMint processing.

Compute individual cap (reused from InvestMint):

$$Cap_T^{ind} = \left\lfloor \frac{\phi_{0,fp} \cdot M_{T-1}^{liq}}{FPONE} \right\rfloor$$

Initialize remaining realizable map for all agents:

$$S_{rem}^{real}(a) := S_{max}^{real}(a, T) \quad \forall a \in \mathcal{A}_T$$

Initialize total auto-liquidation tracker:

$$\Delta_{auto, total} := 0$$

Scan validated events  $e \in \mathcal{E}_T^{val}$  in lexicographic order by  $id(e)$ .

For each event where  $type(e) \in \{\text{STANDARD}, \text{CONFIRM}\}$ :

For each signer  $a \in Signers(e)$  in lexicographic order:

Compute auto-liquidation amount:

$$\Delta_{auto}(a, e) = \min \left( S_{rem}^{real}(a), Cap_T^{ind}, CapRem_T^{sys} \right)$$

Apply liquidation:

$$E_T(a) := E_T(a) + \Delta_{auto}(a, e)$$

$$S_{rem}^{real}(a) := S_{rem}^{real}(a) - \Delta_{auto}(a, e)$$

$$CapRem_T^{sys} := CapRem_T^{sys} - \Delta_{auto}(a, e)$$

$$\Delta_{auto, total} := \Delta_{auto, total} + \Delta_{auto}(a, e)$$

**Forfeit Rule.** After the scan completes, any remaining  $S_{rem}^{real}(a) > 0$  is permanently forfeited (does not carry forward to next epoch).

**Liquid Supply Update (Final).** After auto-liquidation completes:

$$M_T^{liq} := M_T^{liq} + \Delta_{auto,total}$$

This adds auto-liquidated stasis to the liquid supply computed earlier.

**Pipeline Integration.** Auto-liquidation occurs in the Canonical Epoch Pipeline:

- After explicit InvestMint processing (Step 7)
- After Stasis Realization (Step 8)
- Before Income Allocation (Step 10)

The scan order (lexicographic by event ID, then by signer) ensures deterministic liquidation when caps are exhausted.

## 35 Regulator Layer

### 35.1 Branching Degree

Let:

$$n_T := |\mathcal{H}_T|.$$

If  $n_T = 0$ :

$$\sigma_T^{fp} := 0.$$

Otherwise:

$$C_T = \sum_{e \in \mathcal{H}_T} |\text{Children}(e)|.$$

$$\sigma_T^{fp} = \left\lfloor \frac{C_T \cdot \text{FPONE}}{n_T} \right\rfloor.$$

All summations and multiplications MUST be i128-checked.

Branching degree measures average graph connectivity.

### 35.2 Smoothed Branching

Let  $\hat{\sigma}_{T-1}^{fp}$  be stored in  $\mathcal{S}_{T-1}$ .

$$\hat{\sigma}_T^{fp} = \hat{\sigma}_{T-1}^{fp} + \text{mul\_fp}(\beta_{fp}, \sigma_T^{fp} - \hat{\sigma}_{T-1}^{fp}).$$

This is algebraically equivalent to EMA form:

$$\hat{\sigma}_T^{fp} = (1 - \beta_{fp})\hat{\sigma}_{T-1}^{fp} + \beta_{fp}\sigma_T^{fp}$$

but avoids base-integer subtraction in fixed-point.

No floating arithmetic is allowed.

### 35.3 Elasticity Update

Let  $\epsilon_{T-1}^{fp}$  be stored in  $\mathcal{S}_{T-1}$ .

Compute deviation:

$$\Delta_T^{fp} = \sigma_{target,fp} - \hat{\sigma}_{T-1}^{fp}.$$

Compute regulator step:

$$step_T^{fp} = \text{mul\_fp}(\kappa_{P,fp}, \Delta_T^{fp}).$$

Compute pre-clipped elasticity:

$$\tilde{\epsilon}_T^{fp} = \epsilon_{T-1}^{fp} + step_T^{fp}.$$

Apply bounds:

$$\epsilon_T^{fp} = \text{clip}\left(\tilde{\epsilon}_T^{fp}, \epsilon_{\min,fp}, \epsilon_{\max,fp}\right).$$

where:

$$\text{clip}(x, x_{\min}, x_{\max}) = \begin{cases} x_{\min} & \text{if } x < x_{\min} \\ x_{\max} & \text{if } x > x_{\max} \\ x & \text{otherwise} \end{cases}$$

Clipping MUST be deterministic integer comparison.

### 35.4 Regulator Determinism Theorem

**Theorem 5.** *Given valid  $\Theta$  and identical  $(\Theta, \mathcal{L})$ :*

$$\sigma_T^{fp}, \quad \hat{\sigma}_T^{fp}, \quad \epsilon_T^{fp}$$

*are uniquely determined.*

*Proof.* All arithmetic is fixed-point.

All iteration order is canonical (lexicographic over horizon).

All clipping is deterministic integer comparison.

Overflow is statically prevented by -Lint.

Therefore outputs are unique.

□

## 36 Economic Module

### 36.1 Module Enablement

The Economic Module is always enabled in TOP v1.0.

All SKU maps are dense over  $\mathcal{G}$ .

All planet maps are dense over  $\mathcal{P}$ .

Empty universes ( $N_G = 0$  or  $N_P = 0$ ) are permitted but result in trivial economic state.

### 36.2 Lag Convention (Frozen)

All economic updates at epoch  $T$  use:

- Demand and confirmed supply from epoch  $T$  (current)
- Price update uses  $\Phi_{T-1}^{fp}$  (lagged pressure)
- Workforce update uses  $\pi_{T-1}^{fp}$  (lagged preference)
- Planetary multiplier uses  $X_{T-1}^{fp}$  (lagged stress)

No read-after-write within the same epoch.

This lag convention prevents circular dependencies.

### 36.3 Backlog and Supply Update

For each  $g \in \mathcal{G}$ :

Compute epoch demand:

$$D_T(g) = \sum_{\substack{e \in \mathcal{E}_T^{val} \\ type(e)=DEMAND \\ sku(e)=g}} quantity(e)$$

Compute epoch supply:

$$S_T(g) = \sum_{\substack{e \in \mathcal{E}_T^{val} \\ type(e)=CONFIRM \\ sku(e)=g}} quantity(e)$$

All quantities are base integers (i128).

Update backlog ( $B_T(g)$  is i128):

$$B_T(g) = \max \left( 0, \left\lfloor \frac{(\text{FPONE} - \delta_{B,fp}) \cdot B_{T-1}(g)}{\text{FPONE}} \right\rfloor + D_T(g) - S_T(g) \right)$$

Backlog decays geometrically and accumulates net demand.

Update smoothed supply ( $\bar{S}_T(g)$  is i128):

$$\bar{S}_T(g) = \left\lfloor \frac{(\text{FPONE} - \xi_{fp}) \cdot \bar{S}_{T-1}(g) + \xi_{fp} \cdot S_T(g)}{\text{FPONE}} \right\rfloor$$

Smoothed supply is EMA of confirmed supply.

All arithmetic MUST be i128-checked.

### 36.4 Scarcity

For each  $g \in \mathcal{G}$ :

$$\chi_T^{fp}(g) = \text{div\_fp}(B_T(g) \cdot \text{FPONE}, \bar{S}_T(g) \cdot \text{FPONE} + \varepsilon_{\text{num},fp}).$$

Scarcity is the ratio of backlog to smoothed supply.

No conditional branch for zero supply (stabilizer prevents division by zero).

### 36.5 Capacity and Utilization

For each  $g \in \mathcal{G}$ :

Compute capacity:

$$O_T^{fp}(g) = \text{mul\_fp}(A_{fp}(g), L_T^{fp}(g)).$$

Capacity is productivity times workforce allocation.

Compute utilization:

$$u_T^{fp}(g) = \text{div\_fp}(S_T(g) \cdot \text{FPONE}, O_T^{fp}(g) + \varepsilon_{\text{num},fp}).$$

Utilization is the ratio of actual supply to capacity.

### 36.6 Pressure

For each  $g \in \mathcal{G}$ :

$$\Phi_T^{fp}(g) = \chi_T^{fp}(g) - \text{mul\_fp}(\lambda_{u,fp}, \text{FPONE} - u_T^{fp}(g)).$$

Pressure combines scarcity (upward pressure) with underutilization penalty (downward pressure).

### 36.7 Planetary Stress

For each  $p \in \mathcal{P}$ :

Compute impact term:

$$Term_T^{fp}(p) = \sum_{g \in \mathcal{G}} \text{mul\_fp}\left(S_T(g) \cdot \text{FPONE}, \text{imp}_{fp}(g, p)\right)$$

where  $\text{imp}_{fp}(g, p)$  is read from the nested map in  $\Theta$ .

Update planetary stress:

$$X_T^{fp}(p) = \text{mul\_fp}(\text{FPONE} - \zeta_{fp}, X_{T-1}^{fp}(p)) + \text{mul\_fp}(\zeta_{fp}, Term_T^{fp}(p)).$$

Planetary stress is EMA of supply-weighted impacts.

### 36.8 Planetary Multiplier

For each  $g \in \mathcal{G}$ :

Compute stress contribution:

$$StressContrib^{fp}(g) = \sum_{p \in \mathcal{P}} \text{mul\_fp}(X_{T-1}^{fp}(p), \text{imp}_{fp}(g, p))$$

Compute planetary multiplier:

$$\mu_T^{fp}(g) = \text{clip}\left(\text{FPONE} + \text{mul\_fp}(\kappa_{X,fp}, StressContrib^{fp}(g)), \text{FPONE}, \mu_{\max,fp}\right).$$

Multiplier is clamped to  $[\text{FPONE}, \mu_{\max,fp}]$ .

Cannot be less than 1.0 (no negative planetary feedback).

### 36.9 Log-Price Update

For each  $g \in \mathcal{G}$ :

Update log-price:

$$\ell_T^{fp}(g) = \text{clip}\left(\ell_{T-1}^{fp}(g) + \text{mul\_fp}(\kappa_{p,fp}, \Phi_{T-1}^{fp}(g)) + \log_{fp}(\mu_{T-1}^{fp}(g)), \ell_{\min,fp}, \ell_{\max,fp}\right).$$

Log-price evolves based on:

- Lagged pressure (scarcity and utilization)
- Lagged planetary multiplier

Compute price:

$$p_T^{fp}(g) = \exp_{fp}(\ell_T^{fp}(g)).$$

Price is the exponential of log-price.

### 36.10 Workforce Allocation

For each  $g \in \mathcal{G}$ :

Compute workforce utility:

$$a_T^{fp}(g) = \ell_T^{fp}(g) + \text{mul\_fp}(\alpha_{\chi,fp}, \chi_T^{fp}(g))$$

Utility combines price (income potential) with scarcity (demand signal).

Compute max utility for stabilization:

$$m_T = \max_{g \in \mathcal{G}} a_T^{fp}(g)$$

Compute softmax numerator:

$$N_T^{fp}(g) = \exp_{fp}(a_T^{fp}(g) - m_T)$$

Max-subtraction prevents overflow in exponentials.

Compute softmax denominator:

$$\Pi_T^{fp} = \sum_{h \in \mathcal{G}} N_T^{fp}(h) + \varepsilon_{\text{num},fp}$$

Denominator is strictly positive by construction.

No runtime branch permitted.

Compute allocation preference:

$$\pi_T^{fp}(g) = \text{div\_fp}(N_T^{fp}(g), \Pi_T^{fp})$$

Preference is softmax over utilities.

Update workforce allocation:

$$L_T^{fp}(g) = \text{mul\_fp}(\text{FPONE} - \nu_{fp}, L_{T-1}^{fp}(g)) + \text{mul\_fp}(\nu_{fp}, \pi_{T-1}^{fp}(g)).$$

Workforce migrates gradually toward preferred allocations.

**Normalization Rule.**

$$\sum_{g \in \mathcal{G}} L_T^{fp}(g) = \text{FPONE}.$$

Any rounding remainder MUST be assigned to the lexicographically smallest  $g$ .

This ensures workforce is always fully allocated.



### 36.11 Economic Determinism Theorem

**Theorem 6.** *Given identical  $(\Theta, \mathcal{L})$ :*

$$B_T(g), \quad \bar{S}_T(g), \quad p_T^{fp}(g), \quad L_T^{fp}(g), \quad X_T^{fp}(p)$$

*are uniquely determined.*

*Proof.* All universes are dense and finite.

All iteration order is lexicographic.

All arithmetic is fixed-point.

All rounding is floor.

All clipping is deterministic.

Impact matrix  $imp_{fp}(g, p)$  is deterministically read from  $\Theta$ .

Normalization remainder assignment is deterministic (lexicographic smallest).

Therefore all outputs are unique.

□

\*\*\*ab hier mögliche ungenauigkeit\*\*\*

## 37 Canonical Serialization and Hash Contract

### 37.1 Hash Function

The canonical state hash is:

$$H_T = \text{SHA256}(\text{SerializeState}(\mathcal{S}_T)).$$

- Hash algorithm: SHA-256
- Output: raw 32-byte digest
- Text rendering (if displayed): lowercase hexadecimal (non-consensus)

The hash algorithm is consensus-critical.

No alternative hash function is permitted.

### 37.2 Canonical SerializeState Definition

**Definition 10** (Canonical State Bytes). *The canonical serialized state bytes are:*

$$\text{SerializeState}(\mathcal{S}_T) := \text{Header}(T) \| H_\Theta \| \text{Body}(\mathcal{S}_T)$$

where  $H_\Theta$  is the raw 32-byte SHA-256 digest of  $\text{SerializeTheta}(\Theta)$ .

Note:  $H_\Theta$  is raw bytes, not hex-encoded.

**Definition 11** (Header Bytes).

$$\text{Header}(T) := \text{bytes}(\text{"TOPv1.0\#"}) \parallel \text{u32LE}(1) \parallel \text{u32LE}(T)$$

where:

- *Magic string: "TOPv1.0#" (8 ASCII bytes)*
- *Serialization version: 1 (u32 little-endian)*
- *Epoch index: T (u32 little-endian)*

Total header size:  $8 + 4 + 4 + 32 = 48$  bytes (including  $H_\Theta$ ).

## 38 Primitive Encodings

### 38.1 i128 Encoding

$$\text{i128LE}(x)$$

- Signed 128-bit integer
- Two's complement representation
- Little-endian byte order
- Valid range:  $[-2^{127}, 2^{127} - 1]$
- Size: 16 bytes

Overflow MUST abort deterministically.

No silent wrapping is permitted.

### 38.2 u32 Encoding

$$\text{u32LE}(n)$$

- Unsigned 32-bit integer
- Little-endian byte order
- Valid range:  $[0, 2^{32} - 1]$
- Size: 4 bytes

Used exclusively for length prefixes and epoch index.

### 38.3 u8 Encoding

$\text{u8}(b)$

- Unsigned 8-bit integer
- Valid range:  $[0, 255]$
- Size: 1 byte

Used for type tags.

### 38.4 String Encoding

$\text{str}(s) = \text{u32LE}(|s|) \parallel \text{bytes}(s)$

where  $|s|$  is the byte length of string  $s$ .

#### String Constraints.

- ASCII only (all bytes  $b$  satisfy  $0x00 \leq b \leq 0x7F$ )
- No null terminator
- No Unicode normalization
- No trimming
- No case folding

**Recommended Identifier Subset.** For SKU IDs, agent IDs, planet keys, and event IDs:

Restrict to printable ASCII:  $0x20 \leq b \leq 0x7E$ .

This is a recommendation, not a consensus requirement.

### 38.5 Bytes Encoding

$\text{bytes}(s)$

Raw byte sequence of ASCII string  $s$ .

Every byte **MUST** satisfy  $0x00 \leq b \leq 0x7F$  when used for consensus-critical strings.

No encoding transformation is permitted.

## 39 Dense Map Encoding Rule

All consensus-critical maps are dense over a predefined universe.

### 39.1 Canonical Dense Map Encoding

Let  $\mathcal{U} = [u_1, \dots, u_N]$  be a finite universe list in canonical lexicographic order.

A dense map  $f : \mathcal{U} \rightarrow \mathbb{Z}$  MUST be encoded as:

$$\text{u32LE}(N) \| (\text{str}(u_1) \| \text{i128LE}(f(u_1))) \| (\text{str}(u_2) \| \text{i128LE}(f(u_2))) \cdots \| (\text{str}(u_N) \| \text{i128LE}(f(u_N)))$$

#### Requirements.

- Keys MUST appear exactly once
- Keys MUST appear in universe lexicographic order
- All keys in the universe MUST appear
- Missing keys are forbidden
- Duplicate keys are forbidden
- Zero values MUST be serialized explicitly

This resolves all sparse/dense ambiguity.

## 40 Theta Binding

$$H_{\Theta} = \text{SHA256}(\text{SerializeTheta}(\Theta))$$

$H_{\Theta}$  is included in every serialized state.

Nodes MUST reject execution if local  $\Theta$  does not produce the published  $H_{\Theta}$ .

This binds the protocol state to its parameter set.

## 41 SerializeTheta

### 41.1 Structure

$$\text{SerializeTheta}(\Theta) = \text{u32LE}(N_{\text{params}}) \| \text{param}_1 \| \text{param}_2 \cdots \| \text{param}_{N_{\text{params}}}$$

Parameters are sorted lexicographically by key (ASCII string comparison).

### 41.2 Parameter Encoding

Each parameter is encoded as:

$$\text{param}_i = \text{str}(\text{key}) \| \text{type\_tag} \| \text{value}$$

### 41.3 Type Tags

- 0x01 = scalar i128
- 0x02 = u64
- 0x03 = string
- 0x04 = list(i128)
- 0x05 = map(str → i128)

### 41.4 Type-Specific Value Encodings

#### Scalar i128 (tag 0x01)

`value = i128LE(x)`

#### u64 (tag 0x02)

`value = u64LE(n)`

(8 bytes, little-endian)

#### String (tag 0x03)

`value = str(s)`

#### List of i128 (tag 0x04)

`value = u32LE(N) || i128LE(v1) || i128LE(v2) || ... || i128LE(vN)`

Used for DAC lookup tables and universe lists.

#### Map (tag 0x05)

`value = u32LE(N) || (str(k1) || i128LE(v1)) || (str(k2) || i128LE(v2)) || ... || (str(kN) || i128LE(vN))`

Keys MUST be sorted lexicographically.

### 41.5 Forbidden Formats

- No JSON
- No YAML
- No XML
- No whitespace padding
- No comments
- No metadata

## 42 SerializeState Body

The body of  $\text{SerializeState}(\mathcal{S}_T)$  consists of the following sections in fixed order. Order is consensus-critical and immutable.

### 42.1 Section 1: Global Scalars

$$\text{i128LE}(\epsilon_T^{fp}) \parallel \text{i128LE}(\hat{\sigma}_T^{fp}) \parallel \text{i128LE}(M_T^{liq})$$

where:

- $\epsilon_T^{fp}$  is regulator elasticity (fixed-point)
- $\hat{\sigma}_T^{fp}$  is smoothed branching (fixed-point)
- $M_T^{liq}$  is total liquid supply (base integer)

Total size: 48 bytes.

### 42.2 Section 2: Agent Table

Dense over  $\mathcal{A}_T$  (all signers in  $\mathcal{L}$  up to epoch  $T$ ).

$$\text{u32LE}(|\mathcal{A}_T|) \parallel \text{AgentRec}(a_1) \parallel \text{AgentRec}(a_2) \parallel \dots \parallel \text{AgentRec}(a_{|\mathcal{A}_T|})$$

where agents are sorted lexicographically by  $id(a)$ .

#### Agent Record.

$$\text{AgentRec}(a) = \text{str}(id(a)) \parallel \text{i128LE}(E_T(a)) \parallel \text{i128LE}(S_T^{native}(a)) \parallel \text{i128LE}(S_T^{coll}(a)) \parallel \text{i128LE}(rest_T(a))$$

Fields:

- $id(a)$  — agent identifier (ASCII string)
- $E_T(a)$  — liquid balance
- $S_T^{native}(a)$  — native stasis
- $S_T^{coll}(a)$  — collateral stasis
- $rest_T(a)$  — rest counter

### 42.3 Section 3: Horizon ID List

$$\text{u32LE}(|\mathcal{H}_T|) \parallel \text{str}(id_1) \parallel \text{str}(id_2) \parallel \dots \parallel \text{str}(id_{|\mathcal{H}_T|})$$

where IDs are sorted lexicographically.

This list defines the canonical horizon event set.

## 42.4 Section 4: Horizon Event Table

Dense over  $\mathcal{H}_T$ .

$$\mathbf{u32LE}(|\mathcal{H}_T|) \parallel \mathbf{EventRec}(e_1) \parallel \mathbf{EventRec}(e_2) \parallel \dots \parallel \mathbf{EventRec}(e_{|\mathcal{H}_T|})$$

where events are sorted lexicographically by  $id(e)$ .

### Event Record.

$$\mathbf{EventRec}(e) = \mathbf{str}(id(e)) \parallel \mathbf{i128LE}(R_T^{fp}(e)) \parallel \mathbf{i128LE}(DCD_T^{fp}(e)) \parallel \mathbf{i128LE}(\alpha_T^{stored}(e))$$

Fields:

- $id(e)$  — event identifier (SHA-256 hex string)
- $R_T^{fp}(e)$  — ripple value (fixed-point)
- $DCD_T^{fp}(e)$  — distributed contribution density (fixed-point)
- $\alpha_T^{stored}(e)$  — attribution for next epoch (fixed-point)

## 42.5 Section 5: Economic Tables

Each table is encoded as a dense map over its universe.

Tables appear in fixed order:

1.  $B_T(g)$  — backlog (dense over  $\mathcal{G}$ )
2.  $\bar{S}_T(g)$  — smoothed supply (dense over  $\mathcal{G}$ )
3.  $\ell_T^{fp}(g)$  — log-price (dense over  $\mathcal{G}$ )
4.  $p_T^{fp}(g)$  — price (dense over  $\mathcal{G}$ )
5.  $L_T^{fp}(g)$  — workforce allocation (dense over  $\mathcal{G}$ )
6.  $X_T^{fp}(p)$  — planetary stress (dense over  $\mathcal{P}$ )

Each encoded as:

$$\mathbf{u32LE}(|\mathcal{U}|) \parallel (\mathbf{str}(key_1) \parallel \mathbf{i128LE}(value_1)) \parallel \dots \parallel (\mathbf{str}(key_{|\mathcal{U}|}) \parallel \mathbf{i128LE}(value_{|\mathcal{U}|}))$$

Keys are in lexicographic order as defined in  $\Theta$ .

## 42.6 Economic Module Always Enabled

This Audition Ready is the fully integrated TOP v1.0 specification.

Therefore the Economic Module is always enabled.

All economic tables MUST be serialized as dense maps over  $\mathcal{G}$  and  $\mathcal{P}$  as defined in  $\Theta$ .

Empty universes ( $N_G = 0$  or  $N_P = 0$ ) result in empty dense maps (serialized as  $\mathbf{u32LE}(0)$ ).

## 43 Serialization Determinism Theorem

**Theorem 7.** *Given identical  $(\Theta, \mathcal{L})$ , all conformant implementations produce identical:*

$$\text{SerializeState}(\mathcal{S}_T) \quad \text{and} \quad H_T.$$

*Proof.* All key universes are fixed and explicit.

All iteration order is lexicographic.

All fields are mandatory.

All integers are i128 (or u32 for lengths).

No optional fields exist.

No platform-dependent behavior exists.

Byte-level encoding is fully specified.

SHA-256 is deterministic.

Therefore serialization is unique.

□

## 44 -Lint: Overflow Prevention

### 44.1 Purpose

Deterministic abort-on-overflow is necessary but insufficient.

$\Theta$  MUST satisfy static bounds guaranteeing that all intermediate arithmetic fits in i128.

If  $\Theta$  fails lint, the protocol instance is invalid.

Nodes MUST refuse execution.

### 44.2 Event Layer Bounds

Require:

$$N_{E,\max} \leq 2^{31}$$

$$N_{sig,\max} \leq 2^{16}$$

$$b_{\max} \leq 2^{120}$$

$$E_{\max} \leq 2^{120}$$

These bounds MUST be explicitly declared in  $\Theta$ .



### 44.3 Mint Bound

Maximum burn per epoch:

$$B_T^{max} = N_{E,max} \cdot b_{max}$$

Require:

$$B_T^{max} < 2^{120}$$

This ensures burn totals fit in i128 with margin.

Maximum mint:

$$M_T^{max} = B_T^{max} + \left\lceil \frac{\epsilon_{max,fp} \cdot B_T^{max}}{FPONE} \right\rceil$$

Require:

$$M_T^{max} < 2^{120}$$

This ensures mint budget fits in i128.

### 44.4 Income Multiplication Bound

Worst-case numerator in income formula:

$$N^{max} = M_T^{max} \cdot P_{max}^{fp}$$

where  $P_{max}^{fp} \in \Theta$  is an explicit bound on event potential magnitude.

Require:

$$N^{max}$$

$$2^{127}$$

This ensures the intermediate product  $M_T \cdot P_T^{fp}(e)$  fits in i128.

If this bound is violated, i128 multiplication WILL overflow.

### 44.5 Ripple Bound

Worst-case ripple (infinite geometric series):

$$R_{max}^{fp}$$

$$b_{max} \cdot FPONE \cdot \frac{1}{1 - \gamma_{fp}/FPONE}$$

Require:

$$R_{max}^{fp}$$

$2^{120}$

If  $\gamma_{fp}$  is close to FPONE, this bound may be violated.

Deployers MUST verify this condition during  $\Theta$  construction.

#### 44.6 Economic Bound

For SKU and planet universes:

$$N_G < 2^{16}, \quad N_P < 2^{16}$$

Maximum map entries MUST fit in u32.

All price, backlog, and supply values MUST remain below  $2^{120}$  under worst-case dynamics.

Deployers MUST verify economic parameters do not cause unbounded growth.

#### 44.7 DAC Domain Bounds

Verify grid spacing is positive:

$$\Delta_{\log,fp} > 0, \quad \Delta_{\exp,fp} > 0$$

Verify table bounds allow safe interpolation:

$$N_{\log} \geq 2, \quad N_{\exp} \geq 2$$

Verify table values fit in i128:

$$|\log\_table[i]| < 2^{127}, \quad |\exp\_table[j]| < 2^{127}$$

#### 44.8 Lint Validity Rule

**Definition 12** (-Lint Validity).  $\Theta$  is valid if and only if all static bounds above are satisfied.

If not, the protocol instance MUST refuse execution.

Lint verification SHOULD occur before genesis state initialization.

A reference lint checker SHOULD be provided as part of the TOP v1.0 implementation package.

#### 44.9 Recommended Lint Checker Output

A lint checker SHOULD output:

- Pass/fail status

- Violated bounds (if any)
- Margin analysis (how close to limits)
- Recommended parameter adjustments (if close to limits)

Example violation report:

LINT FAILURE:

```
Income multiplication bound violated
N_max = M_max * P_max = 2^121 >= 2^127
Recommendation: reduce P_max or epsilon_max
```

#### 44.10 -Lint Theorem

**Theorem 8** (Overflow Prevention). *If  $\Theta$  passes -Lint validation, then all i128 arithmetic operations in the protocol are guaranteed not to overflow under any valid event log  $\mathcal{L}$ .*

*Proof.* Lint bounds are constructed such that:

- All summations are bounded by explicit universe sizes
- All products have explicit worst-case bounds
- All intermediate values are bounded by explicit parameters
- All bounds include safety margin (typically  $< 2^{120}$  for values,  $< 2^{127}$  for products)

Therefore i128 domain  $([-2^{127}, 2^{127} - 1])$  is never exceeded.

□

## 45 Canonical Epoch Execution Pipeline

### 45.1 Purpose

This section defines the exact execution order of all state transitions within a single epoch.

The order is consensus-critical.

Implementations MUST execute steps in this sequence.

Any deviation in ordering constitutes a protocol fork.

### 45.2 Pipeline Overview

For epoch  $T$ , given pre-state  $\mathcal{S}_{T-1}$  and candidate events  $\mathcal{E}_T^{cand}$ :

1. Validation
2. Apply Burn

3. Horizon Construction
4. DAG Traversal
5. Structural Core (Ripple, DCD, Attribution)
6. Mint Budget Computation
7. InvestMint Processing (Explicit)
8. Stasis Updates
9. Auto-Liquidation
10. Income Allocation
11. Collateral Deposit Processing
12. Economic Module Updates
13. Regulator Updates
14. State Finalization

Each step reads from prior step outputs.

No step may read future step outputs (strict pipeline ordering).

### 45.3 Step-by-Step Execution

#### Step 1: Validation

**Input:**  $\mathcal{S}_{T-1}, \mathcal{E}_T^{cand}$

**Process:** Execute Canonical Validation Procedure.

**Output:**  $\mathcal{E}_T^{val}$  (validated events)

**State mutations:** None (validation is read-only)

#### Step 2: Apply Burn

**Input:**  $\mathcal{E}_T^{val}, E_{T-1}(a)$  for all  $a$

**Process:** For each  $e \in \mathcal{E}_T^{val}$  in lexicographic order:

$$E_T(a) \leftarrow E_T(a) - b_a(e) \quad \forall a \in \text{Signers}(e)$$

**Output:** Updated liquid balances  $E_T(a)$

### Step 3: Horizon Construction

**Input:**  $\mathcal{L}$  (up to epoch  $T$ ),  $H$  (horizon parameter)

**Process:**

$$T_{\min} = \max(0, T - H + 1)$$
$$\mathcal{H}_T = \{e \in \mathcal{L} \mid T_{\min} \leq \text{epoch}(e) \leq T\}$$

**Output:**  $\mathcal{H}_T, HIds_T$  (sorted)

### Step 4: DAG Traversal

**Input:**  $\mathcal{H}_T$

**Process:** Execute Canonical Kahn Algorithm

**Output:** RippleOrder (reversed topological sort)

### Step 5: Structural Core

**Input:**  $\mathcal{H}_T$ , RippleOrder,  $\alpha_{T-1}^{stored}(e)$  for  $e \in \mathcal{H}_{T-1}$

**Process:**

1. Compute Temporal Decay:  $\lambda_{T,fp}(e, y)$  for all parent-child pairs
2. Compute Ripple:  $R_T^{fp}(e)$  for all  $e \in \mathcal{H}_T$  in RippleOrder
3. Compute Structural Intensity:  $\sigma_T^{fp}(e)$
4. Compute DCD:  $DCD_T^{fp}(e)$
5. Compute Total Mass:  $Z_T^{fp}$
6. Compute Attribution:  $\alpha_{T+1}^{fp}(e)$
7. Store:  $\alpha_T^{stored}(e) := \alpha_{T+1}^{fp}(e)$

**Output:**  $R_T^{fp}(e), DCD_T^{fp}(e), \alpha_T^{stored}(e)$  for all  $e \in \mathcal{H}_T$

### Step 6: Mint Budget Computation

**Input:**  $\mathcal{E}_T^{val}, \epsilon_{T-1}^{fp}$

**Process:**

$$B_T = \sum_{e \in \mathcal{E}_T^{val}} b(e)$$

$$M_T = B_T + \left\lfloor \frac{\epsilon_{T-1}^{fp} \cdot B_T}{\text{FPONE}} \right\rfloor$$

**Output:**  $B_T, M_T$

### Step 7: InvestMint Processing (Explicit)

**Input:**  $\mathcal{E}_T^{val}, M_T, M_{T-1}^{liq}, S_{T-1}^{native}(a), S_{T-1}^{coll}(a)$

**Process:**

1. Compute system cap:

$$Cap_T^{sys} = \left\lfloor \frac{\theta_{0,fp} \cdot M_T}{\text{FPONE}} \right\rfloor$$

2. Initialize:  $CapRem_T^{sys} := Cap_T^{sys}$

3. Filter:  $\mathcal{M}_T = \{e \in \mathcal{E}_T^{val} \mid type(e) = \text{INVESTMINT}\}$ , sorted by  $id(e)$

4. For each  $m \in \mathcal{M}_T$  in order:

- Check system cap: if  $\Delta S(m) > CapRem_T^{sys}$ , reject
- Compute individual cap:

$$Cap^{ind} = \left\lfloor \frac{\phi_{0,fp} \cdot M_{T-1}^{liq}}{\text{FPONE}} \right\rfloor$$

- Require:  $\Delta S(m) \leq Cap^{ind}$  and  $\Delta S(m) \leq S_{T-1}^{native}(a) + S_{T-1}^{coll}(a)$
- If accepted: deduct stasis, credit liquid, update  $CapRem_T^{sys}$

**Output:** Updated  $S_T^{native}(a), S_T^{coll}(a), E_T(a), CapRem_T^{sys}, \mathcal{M}_T^{acc}$

**Update liquid supply (partial):**

$$M_T^{liq} := M_{T-1}^{liq} + M_T + \sum_{m \in \mathcal{M}_T^{acc}} \Delta S(m) - B_T$$

### Step 8: Stasis Updates

**Input:**  $\mathcal{E}_T^{val}, rest_{T-1}(a), S_T^{native}(a)$  (post-InvestMint),  $DCD_T^{fp}(e), X_{T-1}^{fp}(p)$

**Process:**

1. Update rest counters:  $rest_T(a)$  for all  $a \in \mathcal{A}_T$
2. Accumulate native stasis:

$$S_T^{native}(a) := S_T^{native}(a) + \text{mul\_fp}(\rho_{fp}, \log_{fp}((1 + rest_T(a)) \cdot \text{FPONE}))$$

3. Compute average planetary stress:

$$X_{avg,T}^{fp} = \left\lfloor \frac{\sum_{p \in \mathcal{P}} X_{T-1}^{fp}(p)}{|\mathcal{P}|} \right\rfloor$$

4. Compute dynamic factors:

$$activity_T^{fp} = \text{div\_fp}(M_T \cdot \text{FPONE}, M_{target})$$

$$stress_T^{fp} = \text{FPONE} + X_{avg,T}^{fp}$$

$$\eta_{S,fp,T}^{eff} = \text{mul\_fp}(\eta_{S,0,fp}, \text{div\_fp}(activity_T^{fp}, stress_T^{fp}))$$

5. Compute total DCD:

$$\Sigma_T^{DCD} = \sum_{e \in \mathcal{E}_T^{val}} DCD_T^{fp}(e)$$

6. Compute max realizable:

$$S_{max}^{real}(a, T) = \min(S_T^{native}(a), \text{mul\_fp}(\eta_{S,fp,T}^{eff}, \Sigma_T^{DCD}))$$

7. Deduct from native:

$$S_T^{native}(a) := S_T^{native}(a) - S_{max}^{real}(a, T)$$

8. Initialize remaining realizable map:

$$S_{rem}^{real}(a) := S_{max}^{real}(a, T)$$

**Output:** Updated  $S_T^{native}(a)$ ,  $rest_T(a)$ ,  $S_{rem}^{real}(a)$

**Step 9: Auto-Liquidation**

**Input:**  $\mathcal{E}_T^{val}$ ,  $S_{rem}^{real}(a)$ ,  $CapRem_T^{sys}$ ,  $M_{T-1}^{liq}$

**Process:** Compute individual cap:

$$Cap_T^{ind} = \left\lfloor \frac{\phi_{0,fp} \cdot M_{T-1}^{liq}}{\text{FPONE}} \right\rfloor$$

Initialize:

$$\Delta_{auto,total} := 0$$

Scan  $\mathcal{E}_T^{val}$  in lexicographic order by  $id(e)$ .

For each  $e$  where  $type(e) \in \{\text{STANDARD}, \text{CONFIRM}\}$ :

For each signer  $a \in Signers(e)$  in lexicographic order:

$$\Delta_{auto}(a, e) = \min(S_{rem}^{real}(a), Cap_T^{ind}, CapRem_T^{sys})$$

Apply:

$$\begin{aligned} E_T(a) &:= E_T(a) + \Delta_{auto}(a, e) \\ S_{rem}^{real}(a) &:= S_{rem}^{real}(a) - \Delta_{auto}(a, e) \\ CapRem_T^{sys} &:= CapRem_T^{sys} - \Delta_{auto}(a, e) \\ \Delta_{auto, total} &:= \Delta_{auto, total} + \Delta_{auto}(a, e) \end{aligned}$$

Forfeit remaining: Any  $S_{rem}^{real}(a) > 0$  after scan is permanently lost.

**Output:** Updated  $E_T(a)$ ,  $\Delta_{auto, total}$

**Update liquid supply (final):**

$$M_T^{liq} := M_T^{liq} + \Delta_{auto, total}$$

**Step 10: Income Allocation**

**Input:**  $\mathcal{E}_T^{val}$ ,  $DCD_T^{fp}(e)$ ,  $M_T$

**Process:**

1. Compute event potential:  $P_T^{fp}(e)$  for all  $e \in \mathcal{E}_T^{val}$

2. Compute total potential:

$$\Sigma_T^{fp} = \sum_{e \in \mathcal{E}_T^{val}} P_T^{fp}(e)$$

3. Compute denominator:

$$Den_T^{fp} = \Sigma_T^{fp} + \varepsilon_{num, fp}$$

4. Compute income:

$$N := M_T \cdot P_T^{fp}(e), \quad I_T(e) := \left\lfloor \frac{N}{Den_T^{fp}} \right\rfloor$$

5. Split income:  $I_{a_i}(e)$  for each signer (lexicographic first gets remainder)

6. Credit balances:

$$E_T(a) := E_T(a) + I_{a_i}(e)$$

**Output:** Updated  $E_T(a)$

**Step 11: Collateral Deposit Processing**

**Input:**  $\mathcal{E}_T^{val}$ ,  $S_T^{coll}(a)$



**Process:** For each  $e \in \mathcal{E}_T^{val}$  where  $type(e) = \text{COLLATERAL\_DEPOSIT}$ :

$$S_T^{coll}(a) := S_T^{coll}(a) + deposit(e)$$

**Output:** Updated  $S_T^{coll}(a)$

## Step 12: Economic Module Updates

**Input:**  $\mathcal{E}_T^{val}$ , all economic state from  $T - 1$

**Process (in order):**

1. Compute demand and supply:  $D_T(g), S_T(g)$
2. Update backlog:  $B_T(g)$
3. Update smoothed supply:  $\bar{S}_T(g)$
4. Compute scarcity:  $\chi_T^{fp}(g)$
5. Compute capacity:  $O_T^{fp}(g)$
6. Compute utilization:  $u_T^{fp}(g)$
7. Compute pressure:  $\Phi_T^{fp}(g)$
8. Update planetary stress:  $X_T^{fp}(p)$
9. Compute planetary multiplier:  $\mu_T^{fp}(g)$
10. Update log-price:  $\ell_T^{fp}(g)$
11. Compute price:  $p_T^{fp}(g)$
12. Compute workforce utility:  $a_T^{fp}(g)$
13. Compute softmax:  $\pi_T^{fp}(g)$
14. Update workforce allocation:  $L_T^{fp}(g)$
15. Apply normalization remainder (to lexicographic smallest)

**Output:** All economic state variables updated

## Step 13: Regulator Updates

**Input:**  $\mathcal{H}_T, \epsilon_{T-1}^{fp}, \hat{\sigma}_{T-1}^{fp}$

**Process:**

1. Compute branching degree:  $\sigma_T^{fp}$
2. Update smoothed branching:  $\hat{\sigma}_T^{fp}$
3. Compute elasticity deviation:  $\Delta_T^{fp}$
4. Update elasticity:  $\epsilon_T^{fp}$

**Output:**  $\epsilon_T^{fp}, \hat{\sigma}_T^{fp}$ **Step 14: State Finalization****Input:** All updated state components**Process:**

1. Finalize agent universe:  $\mathcal{A}_T$  (all signers in  $\mathcal{L}$  up to  $T$ )
2. Produce canonical serialization:  $\text{SerializeState}(\mathcal{S}_T)$
3. Compute state hash:  $H_T = \text{SHA256}(\text{SerializeState}(\mathcal{S}_T))$

**Output:**  $\mathcal{S}_T, H_T$ **45.4 Pipeline Determinism Theorem**

**Theorem 9** (Pipeline Determinism). *Given identical  $(\Theta, \mathcal{L})$  and identical pre-state  $\mathcal{S}_{T-1}$ , all conformant implementations produce identical  $\mathcal{S}_T$  and  $H_T$  when following the Canonical Epoch Execution Pipeline.*

*Proof.* Each step:

- Reads only from prior steps (no circular dependencies)
- Uses deterministic arithmetic (fixed-point, floor rounding)
- Follows explicit ordering rules (lexicographic, topological)
- Contains no conditional branches on tolerances
- Contains no platform-dependent behavior

The pipeline is strictly sequential with explicit data flow.

Therefore the complete pipeline is deterministic.

□

## 45.5 No Alternative Execution Orders

The following alternative orderings are strictly forbidden:

- Processing Income before InvestMint (violates CapRem dependency)
- Processing Auto-Liquidation before Stasis Realization (violates  $S_{max}^{real}$  dependency)
- Processing Economic Module before Income (violates lag convention)
- Processing Regulator before Economic Module (violates planetary stress dependency)
- Processing Auto-Liquidation before explicit InvestMint (violates CapRem sequencing)
- Any parallelization that could introduce race conditions

The pipeline is strictly sequential.

Implementations may optimize internal computations but MUST produce results identical to sequential execution.

## 46 Theorems and Bundle Lock

### 46.1 Protocol Determinism Theorem

**Theorem 10** (Protocol Determinism). *Given identical:*

$$(\Theta, \mathcal{L})$$

*all conformant implementations that follow:*

- *the Canonical Unit System,*
- *the Deterministic Arithmetic Contract (DAC),*
- *the Canonical Event Schema,*
- *the Canonical Validation Procedure,*
- *the Canonical Epoch Execution Pipeline,*
- *the Fixed-Point Core Dynamics,*
- *the Integrated Economic Module,*
- *the Canonical Serialization Contract,*
- *the Overflow-Abort Policy,*
- *the Dense Universe Rules,*
- *the Canonical Step Ordering,*

*produce identical post-state:*

$$S_T \quad \forall T.$$

*Proof.* All iteration order is lexicographically defined.

All numeric domains are sealed (i128, fixed-point).

All rounding is floor.

All clipping is deterministic.

All maps are dense over fixed universes.

No floating-point arithmetic exists.

No platform-dependent behavior exists.

All overflow behavior is abort-deterministic and prevented by -Lint.

The Canonical Epoch Execution Pipeline enforces strict sequencing.

Therefore state transitions are unique.

□

## 46.2 Replay Invariance Theorem

**Theorem 11** (Replay Invariance). *Let two independent nodes execute the protocol with identical  $(\Theta, \mathcal{L})$ .*

*Then:*

$$\mathcal{S}_T^{(1)} = \mathcal{S}_T^{(2)} \quad \forall T$$

*and*

$$H_T^{(1)} = H_T^{(2)} \quad \forall T.$$

*Proof.* From the Protocol Determinism Theorem,  $\mathcal{S}_T$  is uniquely determined.

From the Canonical Serialization Contract,  $\text{SerializeState}(\mathcal{S}_T)$  is uniquely determined.

From SHA-256 determinism,  $H_T$  is uniquely determined.

Therefore replay invariance holds.

□

## 46.3 No Hidden Degrees of Freedom

The protocol contains:

- No implicit defaults
- No heuristic ordering
- No floating arithmetic
- No unit ambiguity
- No alternative rounding

- No sparse map ambiguity
- No alternative burn semantics
- No alternative income semantics
- No alternative topological traversal
- No optional serialization fields
- No silent parameter substitution
- No execution order flexibility

Any deviation constitutes protocol divergence and fork.

#### 46.4 Bundle Lock Declaration

The following components together constitute the complete and exclusive Audition Ready for TOP v1.0:

- Canonical Unit System
- Deterministic Arithmetic Contract (DAC)
- Canonical Event Schema
- Canonical Event Model
- Agent Universe and Admission Rules
- Canonical Validation with Burn Reservation
- Canonical Epoch Execution Pipeline
- Horizon Construction
- Canonical DAG Traversal (Kahn Algorithm)
- Fixed-Point Core Dynamics
- Deterministic Burn Budget and Mint Budget
- InvestMint Processing
- Stasis Mechanism (Accumulation, Realization, Auto-Liquidation)
- Canonical Income Allocation (single form)
- Collateral Deposit Processing
- Integrated Economic Module (always enabled)
- Regulator Layer
- Canonical Serialization and Hash Contract
- $\Theta$  Lint Requirements
- Determinism and Replay Invariance Theorems

This bundle is immutable.

## 46.5 Version Tag

### TOP v1.0 — Audition Ready (Bundle Locked)

Any modification to:

- transition equations,
- arithmetic primitives,
- overflow rules,
- lag conventions,
- execution pipeline ordering,
- serialization layout,
- hash function,
- unit definitions,
- event validation,
- parameter typing,

requires:

1. Version increment (e.g., v3.2),
2. New canonical Audition Ready document,
3. New  $\Theta$  commitment,
4. New golden test vectors.

## 46.6 Final Declaration

The protocol defined in this document is:

- Fully deterministic,
- Fully replay-invariant,
- Distributed-ready,
- Numerically sealed,
- Serialization-sealed,
- Pipeline-sealed,
- Consensus-complete.

No further ambiguity remains.

No interpretive freedom exists.

## 47 Audition Ready Master Context (Handover-Ready)

### 47.1 Purpose of This Section

This section is handover-normative meta-context.

It introduces no mechanics and no new parameters.

It binds interpretation, document precedence, and the remaining work agenda.

### 47.2 Supersession Clause

**Supersession.** This Audition Ready is the exclusive consensus-critical specification of TOP v1.0.

It supersedes and replaces all prior drafts, patches, fragments, engineering notes, and intermediate documents.

**Single Authority.** If any external document conflicts with this Audition Ready, the external document is invalid for v1.0 conformance.

### 47.3 Consensus Boundary vs Tooling Boundary

**Consensus-Critical.** The following are consensus-critical:

- the canonical input  $(\Theta, \mathcal{L})$  definition,
- event validation rules (including burn reservation, signer rules),
- epoch execution pipeline ordering,
- epoch transition equations and canonical ordering,
- all rounding, truncation, clipping, and overflow-abort rules,
- DAC primitive definitions and their parameter binding in  $\Theta$ ,
- canonical state serialization bytes and state hash  $H_T$ ,
- canonical  $\Theta$  serialization bytes and parameter hash  $H_\Theta$ ,
- $\Theta$ -Lint validity (static bounds required to prevent overflow).

**Non-Consensus Tooling.** The following are explicitly non-consensus:

- YAML formats used as test inputs,
- reference simulators, skeleton code, sweep harnesses,
- dashboards, diagnostics, logging formats, UI choices,
- scenario generators and red-team tooling.

**Rule.** Tooling MAY vary. Consensus MUST NOT vary.

No tooling artifact may redefine any consensus rule in this Audition Ready.

#### 47.4 Canonical Deployment Commitments

**$\Theta$  commitment.** A TOP v1.0 deployment MUST publish the genesis commitment  $H_\Theta$ :

$$H_\Theta = \text{SHA256}(\text{SerializeTheta}(\Theta))$$

Nodes MUST refuse execution if the locally loaded  $\Theta$  does not yield the published  $H_\Theta$ .

**State commitment.** For each epoch  $T$  the protocol produces the canonical state hash:

$$H_T = \text{SHA256}(\text{SerializeState}(\mathcal{S}_T))$$

Any divergence in  $\text{SerializeState}$  or  $H_T$  under identical  $(\Theta, \mathcal{L})$  is a protocol fork.

#### 47.5 Remaining Work (Non-Negotiable for Audit-Grade Closure)

The core semantics are closed by this Audition Ready.

However, three categories of work remain required to reach audit-grade readiness.

These items introduce no new mechanics; they complete verification.

##### (R1) Golden Byte/Hash Test Vectors (Required)

At minimum, provide:

- at least one full-core test vector that asserts exact  $\text{SerializeState}(\mathcal{S}_T)$  bytes (or its SHA-256),
- hashes  $H_T$  for a small set of epochs (golden replay targets),
- DAC grid tests (log/exp/sqrt) that assert exact integer outputs on a published domain grid.

##### (R2) Cross-Implementation Hash Equivalence (Required)

Run cross-language replays (e.g., Python vs Rust vs Go) on:

- multiple deterministic random event logs,
- multiple  $\Theta$  packs (including DAC tables),
- verifying identical  $H_T$  at multiple epochs.



### (R3) Formal Determinism Audit Checklist (Recommended)

Produce a final checklist that verifies:

- no symbol duplication across the Audition Ready,
- no hidden defaults remain,
- all truncations are explicit and canonical,
- all map universes are explicit and consistent with serialization,
- all lag conventions are consistent with state indexing,
- $\Theta$ -Lint bounds imply i128 safety for all intermediates.

#### 47.6 Work Agenda for the Next Session

The next work stage MUST:

- not introduce new mechanics,
- not reinterpret semantics,
- only verify internal consistency, generate golden vectors, run cross-implementation checks,
- produce an audit checklist and final closure report.

**Termination Criterion.** Work is complete when:

$$(\Theta, \mathcal{L}) \Rightarrow \mathcal{S}_T \Rightarrow H_T$$

is verified by golden hashes and cross-language replay with zero divergences.

#### 47.7 Handover Package

To continue work with maximal precision and minimal context dependency, a new auditor requires only:

1. This Audition Ready document (bundle locked)
2.  $\Theta$  packs used for testing (including DAC tables), plus their published  $H_\Theta$
3. Golden test vectors that assert field values and at least one full  $H_T$  hash target
4. A non-consensus reference simulator (optional) used only to generate and compare outputs

No older documents are required if this Audition Ready is internally consistent.

## 48 Final Declaration and Status

### 48.1 Specification Status

This document represents the complete, consensus-critical, and exclusive specification of The Ontological Protocol v1.0.

All semantic gaps identified during development have been closed.

All execution ordering ambiguities have been resolved.

All arithmetic operations are explicitly defined.

All iteration orders are canonical.

All serialization is byte-level exact.

### 48.2 Closure Assessment

**Mechanically Complete:** Yes.

All transition equations, validation rules, and pipeline steps are fully specified.

**Deterministically Complete:** Yes.

All sources of non-determinism have been eliminated or canonicalized.

**Serialization Complete:** Yes.

State and  $\Theta$  serialization are byte-level exact with no ambiguity.

**Audit-Ready:** Pending R1-R3.

Golden test vectors and cross-implementation validation required before declaring "granitfest" (audit-grade) status.

### 48.3 Acknowledgment of Philosophical Foundation

The protocol defined here rests on the philosophical framework of Structural Relational Monism as developed in TOP Volumes I and II.

Key principles embedded in this specification:

- **Structural Attribution:** Value emerges from relational position in the causal graph
- **Burn as Thermodynamic Cost:** Irreversible resource expenditure measures commitment
- **Ripple Propagation:** Causal influence flows backward through DAG structure
- **Stasis Mechanism:** Rest and regeneration as ontological necessity, not UBI
- **Economic Integration:** Planetary stress, scarcity, and workforce allocation reflect material constraints

- **Anti-Speculation Design:** No direct price oracle, no extractive rent-seeking

This Audition Ready implements these principles with mathematical precision while remaining philosophically neutral at the bytecode level.

## TOP v1.0 Audition Ready — Bundle Locked

Specification Complete

Golden Test Vectors Required (R1-R3)